

第 4 章 PCIe 总线概述

随着现代处理器技术的发展，在互连领域中，使用高速差分总线替代并行总线是大势所趋。与单端并行信号相比，高速差分信号可以使用更高的时钟频率，使用更少的信号线，完成之前需要许多单端并行数据信号才能达到的总线带宽。

PCI 总线使用并行总线结构，在同一条总线上的所有外部设备共享总线带宽，而 PCIe 总线使用了高速差分总线，并采用端到端的连接方式，因此在每一条 PCIe 链路中只能连接两个设备。这使得 PCIe 与 PCI 总线采用的拓扑结构有所不同。PCIe 总线除了在连接方式上与 PCI 总线不同之外，还使用了一些在网络通信中使用的技术，如支持多种数据路由方式，基于多通路的数据传递方式，和基于报文的数据传送方式，并充分考虑了在数据传送中出现的服务质量 QoS (Quality of Service) 问题。

4.1 PCIe 总线的基础知识

与 PCI 总线不同，PCIe 总线使用端到端的连接方式，在一条 PCIe 链路的两端只能各连接一个设备，这两个设备互为数据发送端和数据接收端。PCIe 总线除了总线链路外，还具有多个层次，发送端发送数据时将通过这些层次，而接收端接收数据时也使用这些层次。PCIe 总线使用的层次结构与网络协议栈较为类似。

4.1.1 端到端的数据传递

PCIe 链路使用“端到端的数据传送方式”，发送端和接收端中都含有 TX（发送逻辑）和 RX（接收逻辑），其结构如图 4-1 所示。

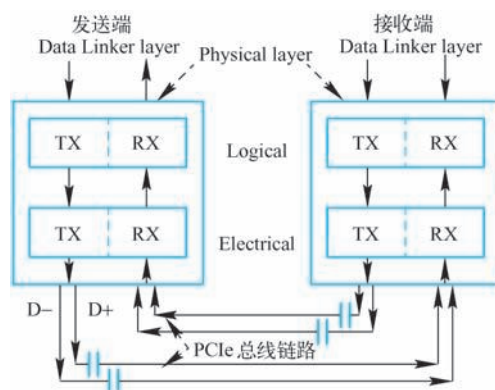


图 4-1 PCIe 总线的物理链路

如上图所示，在 PCIe 总线的物理链路的一个数据通路（Lane）中，有两组差分信号，共 4 根信号线。其中发送端的 TX 部件与接收端的 RX 部件使用一组差分信号连接，该链路也被称为发送端的发送链路，也是接收端的接收链路；而发送端的 RX 部件与接收端的 TX

部件使用另一组差分信号连接，该链路也被称为发送端的接收链路，也是接收端的发送链路。一个 PCIe 链路可以由多个 Lane 组成。

高速差分信号电气规范要求其发送端串接一个电容，以进行 AC 耦合。该电容也被称为 AC 耦合电容。PCIe 链路使用差分信号进行数据传送，一个差分信号由 D+ 和 D- 两根信号组成，信号接收端通过比较这两个信号的差值，判断发送端发送的是逻辑“1”还是逻辑“0”。

与单端信号相比，差分信号抗干扰的能力更强，因为差分信号在布线时要求“等长”“等宽”“贴近”，而且在同层。因此外部干扰噪声将被“同值”而且“同时”加载到 D+ 和 D- 两根信号上，其差值在理想情况下为 0，对信号的逻辑值产生的影响较小。因此差分信号可以使用更高的总线频率。

此外使用差分信号能有效抑制电磁干扰（EMI，Electro Magnetic Interference）。因为差分信号 D+ 与 D- 距离很近而且信号幅值相等、极性相反，这两根线与地线间耦合电磁场的幅值相等，将相互抵消，因此差分信号对外界的电磁干扰较小。当然差分信号的缺点也是显而易见的，一是差分信号使用两根信号传送一位数据；二是差分信号的布线相对严格一些。

PCIe 链路可以由多条 Lane 组成，目前 PCIe 链路可以支持 1、2、4、8、12、16 和 32 个 Lane，即 $\times 1$ 、 $\times 2$ 、 $\times 4$ 、 $\times 8$ 、 $\times 12$ 、 $\times 16$ 和 $\times 32$ 宽度的 PCIe 链路。每一个 Lane 上使用的总线频率与 PCIe 总线使用的版本相关。

第 1 个 PCIe 总线规范为 V1.0，之后依次为 V2.0、V3.0、V4.0、V5.0、V6.0 和 V7.0。目前 PCIe 总线的最新规范为 V7.0。不同的 PCIe 总线规范所定义的总线频率和链路编码方式并不相同，如表 4-1 所示。

表 4-1 PCIe 总线规范与总线频率和链路编码的关系

PCIe 总线规范	总线频率 ^① /GHz	单 Lane 的峰值带宽/(GT/s)	编码方式
V1.x	1.25	2.5	8/10b 编码
V2.x	2.5	5	8/10b 编码
V3.0	4	8	128/130b 编码

① 这里的总线频率指差分信号按照逻辑“0”和“1”连续变化时的频率。

如上表所示，不同的 PCIe 总线规范使用的总线频率并不相同，其使用的数据编码方式也不相同。PCIe 总线 V1.x 和 V2.0 规范在物理层中使用 8/10b 编码，即在 PCIe 链路上的 10 bit 中含有 8 位的有效数据；而 V3.0 规范使用 128/130b 编码方式，即在 PCIe 链路上的 130 bit 中含有 128 位的有效数据。

V3.0 规范使用的总线频率虽然只有 4 GHz，但是其有效带宽是 V2.x 的两倍。有关 8/10b 编码的详细描述见第 7.3.3 节。下文将以 V2.x 规范为例，说明不同宽度 PCIe 链路所能提供的峰值带宽，如表 4-2 所示。

表 4-2 PCIe 总线的峰值带宽

PCIe 总线的数据位宽	$\times 1$	$\times 2$	$\times 4$	$\times 8$	$\times 12$	$\times 16$	$\times 32$
峰值带宽/(GT/s)	5	10	20	40	60	80	160

×32 的 PCIe 链路可以提供 160 GT/s 的链路带宽，远高于 PCI/PCI-X 总线所能提供的峰值带宽。而即将推出的 PCIe V3.0 规范使用 4 GHz 的总线频率，将进一步提高 PCIe 链路的峰值带宽。

在 PCIe 总线中，使用 GT (Gigatransfer) 计算 PCIe 链路的峰值带宽。GT 是在 PCIe 链路上传递的峰值带宽，其计算公式为总线频率×数据位宽×2。

在 PCIe 总线中，影响有效带宽的因素有很多，因而其有效带宽较难计算，这部分内容详见第 12.4.1 节。尽管如此，PCIe 总线提供的有效带宽还是远高于 PCI 总线。PCIe 总线也有其弱点，其中最突出的问题是传送延时。

PCIe 链路使用串行方式进行数据传送，然而在芯片内部，数据总线仍然是并行的，因此 PCIe 链路接口需要进行串并转换，这种串并转换将产生较大的延时。除此之外 PCIe 总线的报文需要经过事务层、数据链路层和物理层，这些报文在穿越这些层次时，也将带来延时。本书将在第 12.4 节详细讨论 PCIe 总线的延时与带宽之间的联系。

在基于 PCIe 总线的设备中，×1 的 PCIe 链路最为常见，而×12 的 PCIe 链路极少出现，×4 和×8 的 PCIe 设备也不多见。Intel 通常在 ICH 中集成了多个×1 的 PCIe 链路用来连接低速外设，而在 MCH 中集成了一个×16 的 PCIe 链路用于连接显卡控制器。而 PowerPC 处理器通常能够支持×8、×4、×2 和×1 的 PCIe 链路。

PCIe 总线物理链路间的数据传送使用基于时钟的同步传送机制，但是在物理链路上并没有时钟线，PCIe 总线的接收端含有时钟恢复模块 CDR (Clock Data Recovery)，CDR 将从接收报文中提取接收时钟，从而进行同步数据传递，PCIe 设备进行链路训练时将完成时钟的提取工作，详见第 8.2 节。

值得注意的是，在一个 PCIe 设备中除了需要从报文中提取时钟外，还使用了 REFCLK+ 和 REFCLK- 信号对作为本地参考时钟，这个信号对的描述见下文。

4.1.2 PCIe 总线使用的信号

PCIe 设备使用两种电源信号供电，分别是 V_{cc} 与 V_{aux} ，其额定电压为 3.3 V。其中 V_{cc} 为主电源，PCIe 设备使用的主要逻辑模块均使用 V_{cc} 供电，而一些与电源管理相关的逻辑使用 V_{aux} 供电。在 PCIe 设备中，一些特殊的寄存器通常使用 V_{aux} 供电，如 Sticky Register，此时即使 PCIe 设备的 V_{cc} 被移除，这些与电源管理相关的逻辑状态和这些特殊寄存器的内容也不会发生改变。

在 PCIe 总线中，使用 V_{aux} 的主要原因是为了降低功耗和缩短系统恢复时间。因为 V_{aux} 在多数情况下并不会被移除，因此当 PCIe 设备的 V_{cc} 恢复后，该设备不必重新恢复使用 V_{aux} 供电的逻辑，从而设备可以很快地恢复到正常工作状态。

PCIe 链路的最大宽度为×32，但是在实际应用中，×32 的链路宽度极少使用。在一个处理器系统中，一般最多提供×16 的 PCIe 插槽，并使用 PETp0~15、PETn0~15 和 PERp0~15、PERn0~15 共 64 根信号线组成 32 对差分信号，其中 16 对 PETxx 信号用于发送链路，另外 16 对 PERxx 信号用于接收链路。这些差分信号的详细说明见第 7.3.1 节。除此之外 PCIe 总线还使用了下列辅助信号。

1. PERST#信号

该信号为全局复位信号，由处理器系统提供，处理器系统需要为 PCIe 插槽和 PCIe 设备

提供该复位信号。PCIe 设备使用该信号复位内部逻辑。当该信号有效时，PCIe 设备将进行复位操作。PCIe 总线定义了多种复位方式，其中 Cold Reset 和 Warm Reset 这两种复位方式的实现与该信号有关，详见第 4.1.5 节。

2. REFCLK+和 REFCLK-信号

在一个处理器系统中，可能含有许多 PCIe 设备，这些设备可以作为 Add-In 卡与 PCIe 插槽连接，也可以作为内置模块，与处理器系统提供的 PCIe 链路直接相连，而不需要经过 PCIe 插槽。PCIe 设备与 PCIe 插槽都具有 REFCLK+和 REFCLK-信号，其中 PCIe 插槽使用这组信号与处理器系统同步。

在一个处理器系统中，通常采用专用逻辑向 PCIe 插槽提供 REFCLK+和 REFCLK-信号，如图 4-2 所示。其中 100 MHz 的时钟源由晶振提供，并经过一个“一推多”的差分时钟驱动器生成多个同相位的时钟源，与 PCIe 插槽一一对应连接。

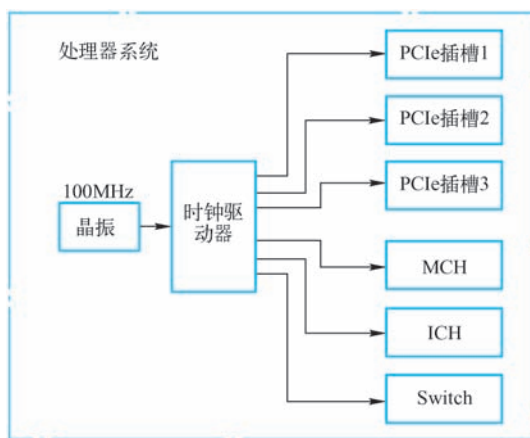


图 4-2 参考时钟与 PCIe 插槽的连接

PCIe 插槽需要使用参考时钟，其频率为 100 MHz。处理器系统需要为每一个 PCIe 插槽、MCH、ICH 和 Switch 提供参考时钟。而且要求在一个处理器系统中，时钟驱动器产生的参考时钟信号到每一个 PCIe 插槽（MCH、ICH 和 Switch）的距离差在 15in 之内。通常信号的传播速度接近光速，约为 6 in/ns，由此可见，不同 PCIe 插槽间 REFCLK+和 REFCLK-信号的传送延时差约为 2.5 ns。

当 PCIe 设备作为 Add-In 卡连接在 PCIe 插槽时，可以直接使用 PCIe 插槽提供的 REFCLK+和 REFCLK-信号，也可以使用独立的参考时钟。内置的 PCIe 设备与 Add-In 卡在处理 REFCLK+和 REFCLK-信号时使用的方法类似，PCIe 设备可以使用独立的参考时钟，而不使用 REFCLK+和 REFCLK-信号。

在 PCIe 设备配置空间的 Link Control Register 中，含有一个“Common Clock Configuration”位。当该位为 1 时，表示该设备与 PCIe 链路的对端设备使用“同相位”的参考时钟；如果为 0，表示该设备与 PCIe 链路的对端设备使用的参考时钟是异步的。Link Control Register 的详细描述见第 4.3.2 节。

在 PCIe 设备中，“Common Clock Configuration”位的缺省值为 0，此时 PCIe 设备使用的参考时钟与对端设备没有任何联系，PCIe 链路两端设备使用的参考时钟可以异步设置。这

个异步时钟设置方法对于使用 PCIe 链路进行远程连接时尤为重要。

在一个处理器系统中, 如果使用 PCIe 链路进行机箱到机箱间的互连, 因为参考时钟可以异步设置, 机箱到机箱之间进行数据传送时仅需要差分信号线即可, 而不需要参考时钟, 从而极大地降低了连接难度。

3. WAKE#信号

当 PCIe 设备进入休眠状态, 主电源已经停止供电时, PCIe 设备使用该信号向处理器系统提交唤醒请求, 使处理器系统重新为该 PCIe 设备提供主电源 V_{cc} 。在 PCIe 总线中, WAKE#信号是可选的, 因此使用 WAKE#信号唤醒 PCIe 设备的机制也是可选的。值得注意的是产生该信号的硬件逻辑必须使用辅助电源 V_{aux} 供电。

WAKE#是一个 Open Drain 信号, 一个处理器的所有 PCIe 设备可以将 WAKE#信号进行线与后, 统一发送给处理器系统的电源控制器。当某个 PCIe 设备需要被唤醒时, 该设备首先置 WAKE#信号有效, 然后在经过一段延时之后, 处理器系统开始为该设备提供主电源 V_{cc} , 并使用 PERST#信号对该设备进行复位操作。此时 WAKE#信号需要始终保持为低, 当主电源 V_{cc} 上电完成之后, PERST#信号也将置为无效并结束复位, WAKE#信号也将随之置为无效, 结束整个唤醒过程。

PCIe 设备除了可以使用 WAKE#信号实现唤醒功能外, 还可以使用 Beacon 信号实现唤醒功能。与 WAKE#信号实现唤醒功能不同, Beacon 使用 In-band 信号, 即差分信号 D+和D-实现唤醒功能。Beacon 信号 DC 平衡, 由一组通过 D+和 D-信号生成的脉冲信号组成。这些脉冲信号宽度的最小值为 2 ns, 最大值为 16 μ s。当 PCIe 设备准备退出 L2 状态 (该状态为 PCIe 设备使用的一种低功耗状态) 时, 可以使用 Beacon 信号, 提交唤醒请求。

4. SMCLK 和 SMDAT 信号

SMCLK 和 SMDAT 信号与 x86 处理器的 SMBus (System Management Bus) 相关。SMBus 于 1995 年由 Intel 提出, SMBus 由 SMCLK 和 SMDAT 信号组成。SMBus 源于 I²C 总线, 但是与 I²C 总线存在一些差异。

SMBus 的最高总线频率为 100 kHz, 而 I²C 总线可以支持 400 kHz 和 2 MHz 的总线频率。此外 SMBus 上的从设备具有超时功能, 当从设备发现主设备发出的时钟信号保持低电平超过 35 ms 时, 将引发从设备的超时复位。在正常情况下, SMBus 的主设备使用的总线频率最低为 10 kHz, 以避免从设备在正常使用过程中出现超时。

在 SMBus 中, 如果主设备需要复位从设备时, 可以使用这种超时机制。而 I²C 总线只能使用硬件信号才能实现这种复位操作, 在 I²C 总线中, 如果从设备出现错误时, 单纯通过主设备是无法复位从设备的。

SMBus 还支持 Alert Response 机制。当从设备产生一个中断时, 并不会立即清除该中断, 直到主设备向 0b0001100 地址发出命令。

上文所述的 SMBus 和 I²C 总线的区别还是局限于物理层和链路层上, 实际上 SMBus 还含有网络层。SMBus 还在网络层上定义了 11 种总线协议, 用来实现报文传递。

SMBus 在 x86 处理器系统中得到了大规模普及, 其主要作用是管理处理器系统的外部设备, 并收集外设的运行信息, 特别是一些与智能电源管理相关的信息。PCI 和 PCIe 插槽也为 SMBus 预留了接口, 以便 PCI/PCIe 设备与处理器系统进行交互。

在 Linux 系统中, SMBus 得到了广泛的应用, ACPI 也为 SMBus 定义了一系列命令, 用

于智能电池、电池充电器与处理器系统之间的通信。在 Windows 操作系统中，有关外部设备的描述信息，也是通过 SMBus 获得的。

5. JTAG 信号

JTAG (Joint Test Action Group) 是一种国际标准测试协议，与 IEEE 1149.1 兼容，主要用于芯片内部测试。目前绝大多数器件都支持 JTAG 测试标准。JTAG 信号由 TRST#、TCK、TDI、TDO 和 TMS 信号组成。其中 TRST# 为复位信号；TCK 为时钟信号；TDI 和 TDO 分别与数据输入和数据输出对应；而 TMS 信号为模式选择。

JTAG 允许多个器件通过 JTAG 接口串联在一起，并形成 JTAG 链。目前 FPGA 和 EPLD 可以借用 JTAG 接口实现在线编程 (In-System Programming, ISP) 功能。处理器也可以使用 JTAG 接口进行系统级调试工作，如设置断点、读取内部寄存器和存储器等一系列操作。除此之外 JTAG 接口也可用作“逆向工程”，分析一个产品的实现细节，因此在正式产品中，一般不保留 JTAG 接口。

6. PRSNT1#和 PRSNT2#信号

PRSNT1#和 PRSNT2#信号与 PCIe 设备的热插拔相关。在基于 PCIe 总线的 Add-In 卡中，PRSNT1#和 PRSNT2#信号直接相连，而在处理器主板中，PRSNT1#信号接地，PRSNT2#信号通过上拉电阻接为高。PCIe 设备的热插拔结构如图 4-3 所示。

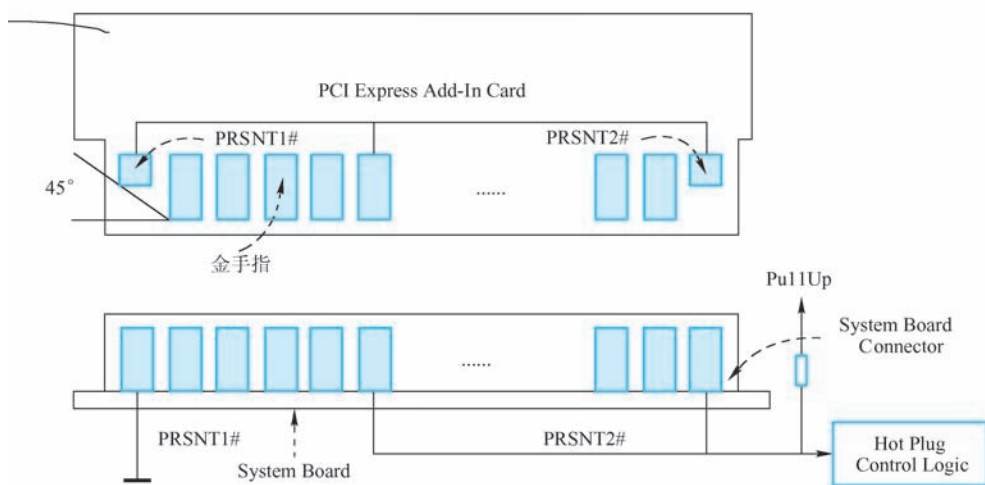


图 4-3 PCIe 设备的热插拔

当 Add-In 卡没有插入时，处理器主板的 PRSNT2#信号由上拉电阻接为高，而当 Add-In 卡插入时主板的 PRSNT2#信号将与 PRSNT1#信号通过 Add-In 卡连通，此时 PRSNT2#信号为低。处理器主板的热插拔控制逻辑将捕获这个“低电平”，得知 Add-In 卡已经插入，从而触发系统软件进行相应处理。

Add-In 卡拔出的工作机制与插入类似。当 Add-In 卡连接在处理器主板时，处理器主板的 PRSNT2#信号为低；当 Add-In 卡拔出后，处理器主板的 PRSNT2#信号为高。处理器主板的热插拔控制逻辑将捕获这个“高电平”，得知 Add-In 卡已经被拔出，从而触发系统软件进行相应处理。

不同的处理器系统处理 PCIe 设备热拔插的过程并不相同，在一个实际的处理器系统中，

热拔插设备的实现也远比图 4-3 中的示例复杂得多。值得注意的是，在实现热拔插功能时，Add-In 卡需要使用“长短针”结构。

如图 4-3 所示，PRSENT1#和 PRSENT2#信号使用的金手指长度是其他信号的一半。因此当 PCIe 设备插入插槽时，PRSENT1#和 PRSENT2#信号在其他金手指与 PCIe 插槽完全接触，并经过一段延时后，才能与插槽完全接触；当 PCIe 设备从 PCIe 插槽中拔出时，这两个信号首先与 PCIe 插槽断开，再经过一段延时后，其他信号才能与插槽断开。系统软件可以使用这段延时，进行一些热拔插处理。

4.1.3 PCIe 总线的层次结构

PCIe 总线采用了串行连接方式，并使用数据包（Packet）进行数据传输，采用这种结构有效去除了在 PCI 总线中存在的一些边带信号，如 INTx 和 PME#等信号。在 PCIe 总线中，数据报文在接收和发送过程中，需要通过多个层次，包括事务层、数据链路层和物理层。PCIe 总线的层次结构如图 4-4 所示。

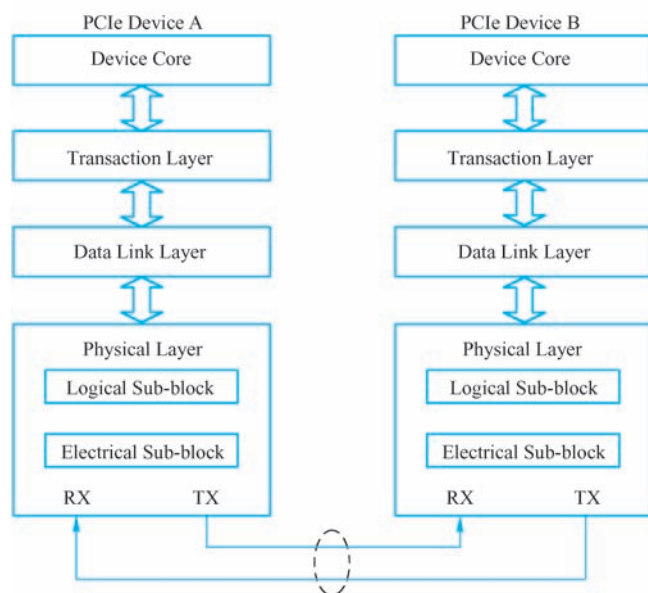


图 4-4 PCIe 总线的层次组成结构

PCIe 总线的层次组成结构与网络中的层次结构有类似之处，但是 PCIe 总线的各个层次都是使用硬件逻辑实现的。在 PCIe 体系结构中，数据报文首先在设备的核心层（Device Core）中产生，然后再经过该设备的事务层（Transaction Layer）、数据链路层（Data Link Layer）和物理层（Physical Layer），最终发送出去。而接收端的数据也需要通过物理层、数据链路和事务层，并最终到达核心层。

1. 事务层

事务层定义了 PCIe 总线使用总线事务，其中多数总线事务与 PCI 总线兼容。这些总线事务可以通过 Switch 等设备传送到其他 PCIe 设备或者 RC。RC 也可以使用这些总线事务访问 PCIe 设备。

事务层接收来自 PCIe 设备核心层的数据，并将其封装为 TLP（Transaction Layer Packet）后发向数据链路层。此外事务层还可以从数据链路层中接收数据报文，然后转发至 PCIe 设备的核心层。

事务层的一个重要工作是处理 PCIe 总线的“序”。在 PCIe 总线中，“序”的概念非常重要，也较难理解。在 PCIe 总线中，事务层传递报文时可以乱序，这为 PCIe 设备的设计制造了不小的麻烦。事务层还使用流量控制机制保证 PCIe 链路的使用效率。有关事务层的详细说明见第 6 章。

2. 数据链路层

数据链路层保证来自发送端事务层的报文可以可靠、完整地发送到接收端的数据链路层。来自事务层的报文在通过数据链路层时，将被添加 Sequence Number 前缀和 CRC 后缀。数据链路层使用 ACK/NAK 协议保证报文的可靠传递。

PCIe 总线的数据链路层还定义了多种 DLLP（Data Link Layer Packet），DLLP 产生于数据链路层，终止于数据链路层。值得注意的是，TLP 与 DLLP 并不相同，DLLP 并不是由 TLP 加上 Sequence Number 前缀和 CRC 后缀组成的。数据链路层的详细描述见第 7 章。

3. 物理层

物理层是 PCIe 总线的最底层，将 PCIe 设备连接在一起。PCIe 总线的物理电气特性决定了 PCIe 链路只能使用端到端的连接方式。PCIe 总线的物理层为 PCIe 设备间的数据通信提供传送介质，为数据传送提供可靠的物理环境。

物理层是 PCIe 体系结构最重要，也是最难以实现的组成部分。PCIe 总线的物理层定义了 LTSSM（Link Training and Status State Machine）状态机，PCIe 链路使用该状态机管理链路状态，并进行链路训练、链路恢复和电源管理。

PCIe 总线的物理层还定义了一些专门的“序列”，有的书籍将物理层这些“序列”称为 PLP（Physical Layer Packet），这些序列用于同步 PCIe 链路，并进行链路管理。值得注意的是 PCIe 设备发送 PLP 与发送 TLP 的过程有所不同。

对于系统软件而言，物理层几乎不可见，但是系统程序员仍有必要较为深入地理解物理层的工作原理。本书将在第 7.3 节详细描述 PCIe 总线物理层的实现机制，并在第 8 章详细介绍 LTSSM 状态机。

4.1.4 PCIe 链路的扩展

PCIe 链路使用端到端的数据传送方式。在一条 PCIe 链路中，这两个端口是完全对等的，分别连接发送与接收设备，而且一个 PCIe 链路的一端只能连接一个发送设备或者接收设备。因此 PCIe 链路必须使用 Switch 扩展 PCIe 链路后，才能连接多个设备。使用 Switch 进行链路扩展的实例如图 4-5 所示。

在 PCIe 总线中，Switch^①是一个特殊的设备，该设备由 1 个上游端口和 2~n 个下游端口组成。PCIe 总线规定，在一个 Switch 中可以与 RC 直接或者间接相连^②的端口为上游端口，

① 本节出现的 Switch 指传统的 Switch，在 MR-IOV 规范定义的 Switch 与此并不相同，详见第 13.3.2 节。

② 所谓间接相连是指通过其他 Switch 再与 RC 相连。

在 PCIe 总线中，RC 的位置一般在上方，这也是上游端口这个称呼的由来。在 Switch 中除了上游端口外，其他所有端口都被称为下游端口。下游端口一般与 EP 相连，或者连接下一级 Switch 继续扩展 PCIe 链路。其中与上游端口相连的 PCIe 链路被称为上游链路，与下游端口相连的 PCIe 链路被称为下游链路。

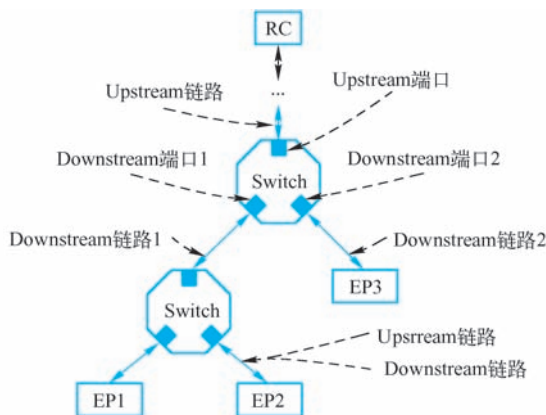


图 4-5 使用 Switch 扩展 PCIe 链路

上游（Upstream）链路和下游（Downstream）链路是相对的概念。如图 4-5 所示，Switch 与 EP2 连接的 PCIe 链路，对于 EP2 而言是上游链路，而对 Switch 而言是下游链路。

图中，Switch 中含有 3 个端口，其中一个为上游端口（Upstream Port），而其他两个为下游端口（Downstream Port）。其中上游端口与 RC 或者其他 Switch 的下游端口相连，而下游端口与 EP 或者其他 Switch 的上游端口相连。

在 Switch 中，还有两个与端口相关的概念，分别是 Egress 端口和 Ingress 端口。这两个端口与通过 Switch 的数据流向有关。其中 Egress 端口指发送端口，即数据离开 Switch 使用的端口；Ingress 端口指接收端口即数据进入 Switch 使用的端口。

Egress 端口和 Ingress 端口与上下游端口没有对应关系。在 Switch 中，上下游端口可以作为 Egress 端口，也可以作为 Ingress 端口。如图 4-5 所示，RC 对 EP3 的内部寄存器进行操作时，Switch 的上游端口为 Ingress 端口，而下游端口为 Egress 端口；当 EP3 对主存储器进行 DMA 写操作时，该 Switch 的上游端口为 Egress 端口，而下游端口为 Ingress 端口。

PCIe 总线还规定了一种特殊的 Switch 连接方式，即 Crosslink 连接模式。支持这种模式的 Switch，其上游端口可以与其他 Switch 的上游端口连接，其下游端口可以与其他 Switch 的下游端口连接。

PCIe 总线提供 CrossLink 连接模式的主要目的是为了解决不同处理器系统之间的互连，如图 4-6 所示。使用 CrossLink 连接模式时，虽然从物理结构上看，一个 Switch 的上/下游端口与另一个 Switch 的上/下游端口直接相连，但是这个 PCIe 链路经过训练后，仍然是一个端口作为上游端口，而另一个端口作为下游端口。

处理器系统 1 与处理器系统 2 间的数据交换可通过 Crosslink 进行。当处理器系统 1(2) 访问的 PCI 总线域的地址空间或者 Requester ID 不在处理器系统 1(2) 内时，这些数据将被 Crosslink 端口接收，并传递到对端处理器系统中。Crosslink 对端接口的 P2P 桥将接收来自另一个处理器域的数据请求，并将其转换为本处理器域的数据请求。

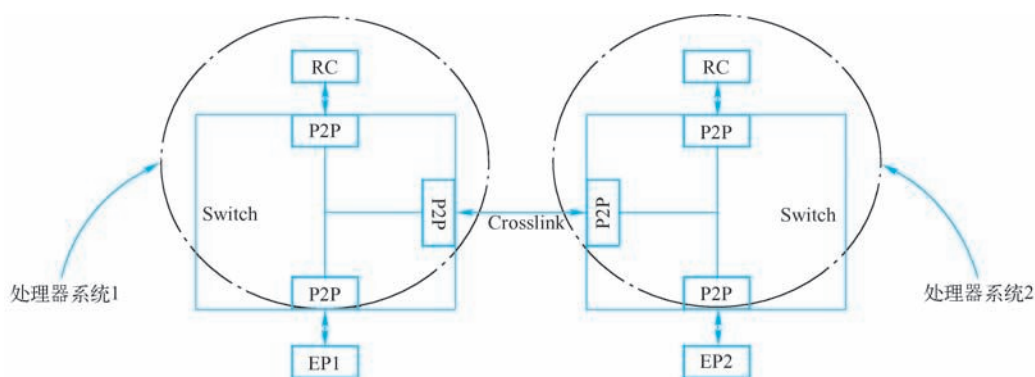


图 4-6 使用 CrossLink 方式连接两个处理器系统

使用 Crosslink 方式连接两个拓扑结构完全相同的处理器系统时，仍然有不足之处。假设图 4-6 中的处理器系统 1 和 2 的 RC 使用的 ID 号都为 0，而主存储器都是从 0x0000-0000 开始编址。当处理器系统 1 读取 EP2 的某段 PCI 总线空间时，EP2 将使用 ID 路由方式，将完成报文传送给 ID 号为 0 的 PCI 设备，此时是处理器系统 2 的 RC 而不是处理器系统 1 的 RC 收到 EP2 的数据。因为处理器系统 1 和 2 的 RC 使用的 ID 号都为 0，EP2 不能区分这两个 RC。

综上所述，使用 Crosslink 方式并不能完全解决两个处理器系统的互连问题，因此在有些 Switch 中支持非透明桥结构。这种结构与 PCI 总线非透明桥的实现机制类似，本章对此不做进一步说明。

使用非透明桥仅解决了两个处理器间数据通路问题，但是不便于 NUMA 结构对外部设备的统一管理。PCIe 总线对此问题的解决方法是使用 MR-IOV 技术，该技术要求 Switch 具有多个上游端口分别与不同的 RC 互连。目前 PLX 公司已经可以提供具有多个上游端口的 Switch，但是尚未实现 MR-IOV 技术涉及的一些与虚拟化相关的技术。有关 MR-IOV 技术的详细说明见第 13.3.2 节。

即便 MR-IOV 技术可以合理解决多个处理器间的数据访问和对 PCIe 设备的配置管理，使用 PCIe 总线进行两个或者多个处理器系统间的数据传递仍然是一个不小问题。因为 PCIe 总线的传送延时仍然是制约其在大规模处理器系统互连中应用的重要因素。

4.1.5 PCIe 设备的初始化

PCIe 总线规定了两大类复位方式，一种是传统的复位方式（Conventional Reset），另一种是 FLR（Function-Level Reset）方式。

其中 PCIe 总线的传统复位方式由两大类组成，一个是 Fundamental Reset，而另一个是 Non-Fundamental Reset。Fundamental Reset 方式包括 Cold 和 Warm Reset 方式，可以将 PCIe 设备中的绝大多数内部寄存器和内部状态都恢复成初始值；而 Non-Fundamental Reset 方式指 Hot Reset 方式。

1. 传统复位方式

传统复位方式分为 Cold、Warm 和 Hot Reset。PCIe 设备可以根据当前设备的运行状态选择合适的复位方式，PCIe 总线提供多种复位方式的主要原因是减小 PCIe 设备的复位延时。

其中传统复位方式的延时大于 FLR 方式。使用传统复位方式时，Cold Reset 使用的时间最长，而 Hot Reset 使用的时间最短。

当一个 PCIe 设备的 V_{cc} 电源上电后，处理器系统将置该设备的 PERST# 信号为有效，此时将引发 PCIe 设备的复位操作，PCIe 总线将这种复位方式称为 Cold Reset 方式。Cold Reset 无疑是一种彻底的复位方式，这种方式属于 Fundamental Reset。PCIe 设备进行 Cold Reset 时，所有使用 V_{cc} 进行供电的寄存器和 PCIe 端口逻辑将无条件进入初始状态。但是使用这种方式依然无法复位使用 V_{aux} 供电的寄存器和逻辑，这些寄存器和逻辑只能在处理器完全下电时才能被彻底复位。

当 PCIe 设备完成上电过程后，也可能重新进行 Fundamental Reset，这种复位方式也被称为 Warm Reset。PCIe 总线并没有规定 Warm Reset 的具体实现方式。一个 PCIe 设备可以使用 Watchdog 逻辑，对该 PCIe 设备进行复位，这种方式就是 Warm Reset 的一种。Warm Reset 也是一种 Fundamental Reset。

除了 Cold 和 Warm Reset 方式外，PCIe 总线还规定了另一种复位方式，即 Hot Reset 方式。当 PCIe 设备出现某种异常时，可以使用软件手段对该设备进行复位。如系统软件将 Bridge Control Register 的 Secondary Bus Reset 位置 1 时，该桥片将 Secondary 总线上的 PCI/PCIe 设备进行 Hot Reset。对于 PCI 总线，当 Secondary Bus Reset 位置 1 时，Secondary 总线将使用 RST# 信号对其下游的 PCI 设备进行复位。而 PCIe 总线将通过 TS1 和 TS2 序列对下游设备进行 Hot Reset。

在 TS1 和 TS2 序列中包含一个 Hot Reset 位。当下游设备收到一个 TS1 和 TS2 序列，而且其 Hot Reset 位为 1 时，下游设备将使用 Hot Reset 方式进行复位操作。有关 TS1 和 TS2 序列的详细描述见第 8.1.1 节。除此之外当数据链路层向事务层报告 DL_Down 时，该设备也将进行 Hot Reset。DL_Down 状态的详细说明见第 7.1.1 节。

Hot Reset 方式并不属于 Fundamental Reset。PCIe 设备进行 Hot Reset 方式时，也可以将 PCIe 设备的多数寄存器和状态恢复为初始值。

当 PCIe 设备完成传统复位方式后，经过一段延时后，将开始进行 PCIe 总线的链路训练，有关链路训练的详细说明见第 8 章。

2. FLR 方式

除了传统复位方式之外，PCIe 总线还提供了 FLR 方式。系统软件填写某些寄存器时，PCIe 设备将使用 FLR 方式进行复位。支持 FLR 方式的 PCIe 设备需要在其 BAR 空间中提供一个寄存器，当系统软件对该寄存器的 Function Level Reset 位写 1 时，PCIe 设备将使用 FLR 方式复位 PCIe 设备的内部逻辑。FLR 方式对于 PCIe 设备是可选的，PCIe 总线规范并没有定义 FLR 方式的具体实现过程，但是定义了 FLR 方式的适用范围。

在一个处理器系统中，如果某个 PCIe 设备出现故障时，系统软件需要停止这个 PCIe 设备的所有 I/O 操作。如一个 PCIe 网卡出现故障时，系统软件需要对这个 PCIe 网卡的“与 PCIe 相关”和“与网络部分相关”的逻辑复位，而 Cold、Warm 和 Hot Reset 无法复位“与网络部分相关”的逻辑。此时需要使用 FLR 方式合理复位这个网卡。有些不支持 FLR 方式的网卡，也可以使用传统方式复位“与网络部分相关”的逻辑。

在一个大规模并行处理器系统中，系统软件使用分区的管理所有硬件资源，包括处理器资源和所有 I/O 资源，在这些 I/O 资源中通常会包含 PCIe 设备。在这种处理器系统中，

任务在指定的分区中运行，当这个任务执行完毕后，系统软件需要调整硬件资源分区。此时受到影响的 PCIe 设备需要使用 FLR 方式复位内部逻辑，以免造成对新分区的资源污染，并保护之前任务的运行结果。

此外当系统软件初始化与某个 PCIe 设备相关的软件协议栈时，也可能需要使用 FLR 方式复位 PCIe 设备内部的逻辑。

由上文所示，PCIe 设备使用的 FLR 方式与传统复位方式有所不同。但是对于一些不支持 FLR 方式的 PCIe 设备，也可以使用传统复位方式实现 FLR 方式。但是采用这种方式，与 FLR 方式相比，PCIe 设备的初始化恢复时间较长。因为传统复位方式几乎复位了所有“与 PCIe 链路相关”的逻辑，而 FLR 方式仅复位部分“与 PCIe 链路相关”的逻辑。在 PCIe 总线中，链路训练与重训练的过程较长。

当 PCIe 设备使用 FLR 方式进行复位时，有些与 PCIe 链路相关的状态和寄存器并不会被复位，如下所示。

- Sticky Registers。与传统复位方式相同，FLR 方式也不能复位这些寄存器，但是系统软件可以对部分 Sticky Registers 进行修改。当 V_{aux} 被移除后，在这些寄存器中保存的数据才会丢失。
- HwInit 类型的寄存器。在 PCIe 设备中，有些配置寄存器的属性为 HwInit，这些寄存器的值由芯片的配置引脚决定，或者在上电复位后从 E²PROM 中获取。Cold 和 Warm Reset 可以复位这些寄存器，然后从 E²PROM 中重新获取数据；但是使用 FLR 方式，不能复位这些寄存器。
- 此外还有一些特殊的配置寄存器不能被 FLR 方式复位，如 Max_Payload_Size、RCB 和一些与电源管理、流量控制和链路控制直接相关的寄存器。
- FLR 方式不会影响 LTSSM 状态机。

4.2 PCIe 体系结构的组成部件

PCIe 总线作为处理器系统的局部总线，其作用与 PCI 总线类似，主要目的是为了连接处理器系统中的外部设备，当然 PCIe 总线也可以连接其他处理器系统。在不同的处理器系统中，PCIe 体系结构的实现方法略有不同。但是在大多数处理器系统中，都使用了 RC、Switch 和 PCIe-to-PCI 桥这些基本模块连接 PCIe 和 PCI 设备。在 PCIe 总线中，基于 PCIe 总线的设备，也称为 EP（Endpoint）。

4.2.1 基于 PCIe 架构的处理器系统

在不同的处理器系统中，PCIe 体系结构的实现方式不尽相同。PCIe 体系结构以 Intel 的 x86 处理器为蓝本实现，已深深地烙下 x86 处理器的印记。在 PCIe 总线规范中，有许多内容是 x86 处理器独有的，也仅在 x86 处理器的 Chipset 中存在。在 PCIe 总线规范中，一些最新的功能也在 Intel 的 Chipset 中率先实现。

本节将以一个虚拟的处理器系统 A 和 PowerPC 处理器为例简要介绍 RC 的实现，并简单归纳 RC 的通用实现机制。

1. 处理器系统 A

在有些处理器系统中，没有直接提供 PCI 总线，此时需要使用 PCIe 桥，将 PCIe 链路转换为 PCI 总线之后，才能连接 PCI 设备。在 PCIe 体系结构中，也存在 PCI 总线号的概念，其编号方式与 PCI 总线兼容。一个基于 PCIe 架构的处理器系统 A 如图 4-7 所示。

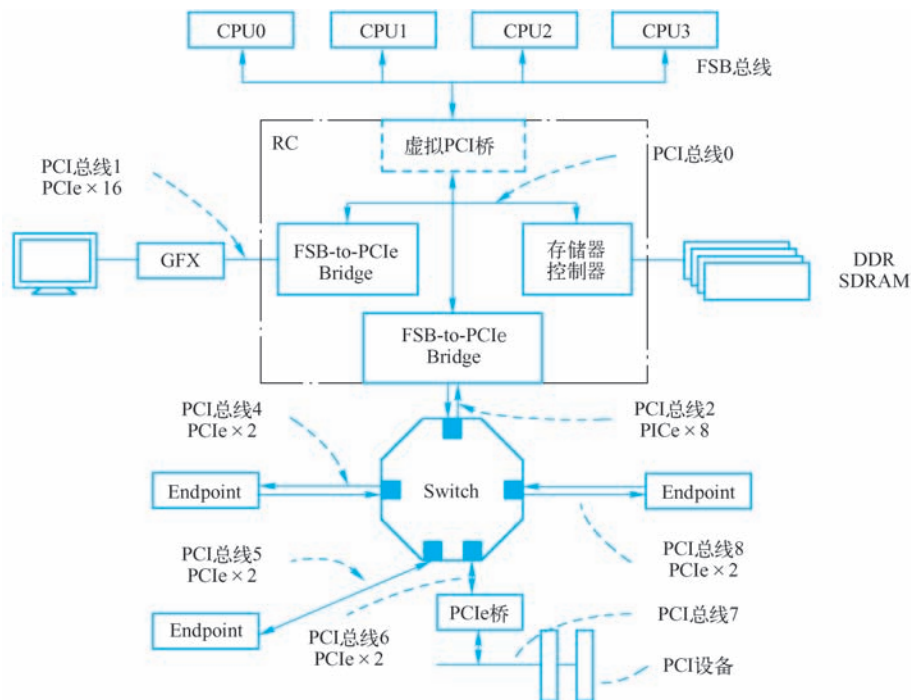


图 4-7 基于 PCIe 架构的处理器系统 A

在图 4-7 的结构中，处理器系统首先使用一个虚拟的 PCI 桥分离处理器系统的存储器域与 PCI 总线域。FSB 总线下的所有外部设备都属于 PCI 总线域。与这个虚拟 PCI 桥直接相连的总线为 PCI 总线 0。这种架构与 Intel 的 x86 处理器系统较为类似。

在这种结构中，RC 由两个 FSB-to-PCIe 桥和存储器控制器组成。值得注意的是在图 4-7 中，虚拟 PCI 桥的作用只是分离存储器域与 PCI 总线域，但是并不会改变信号的电气特性。RC 与处理器通过 FSB 连接，而从电气特性上看，PCI 总线 0 与 FSB 兼容，因此在 PCI 总线 0 上挂接的是 FSB-to-PCIe 桥，而不是 PCI-to-PCIe 桥。

在 PCI 总线 0 上有一个存储器控制器和两个 FSB-to-PCIe 桥。这两个 FSB-to-PCIe 桥分别推出一个 x16 和 x8 的 PCIe 链路，其中 x16 的 PCIe 链路连接显卡控制器（GFX），其编号为 PCI 总线 1；x8 的 PCIe 链路连接一个 Switch 进行 PCIe 链路扩展。而存储器控制器作为 PCI 总线 0 的一个 Agent 设备，连接 DDR 插槽或者颗粒。

此外在这个 PCI 总线上还可能连接了一些使用“PCI 配置空间”管理的设备，这些设备的访问方法与 PCI 总线兼容，在 x86 处理器的 Chipset 中集成了一些内嵌的设备。这些内嵌的设备均使用“PCI 配置空间”进行管理，包括存储器控制器。

PCIe 总线使用端到端的连接方式，因此只有使用 Switch 才能对 PCIe 链路进行扩展，而每扩展一条 PCIe 链路将产生一个新的 PCI 总线号。如图 4-7 所示，Switch 可以将 1 个 x8 的

PCIe 端口扩展为 4 个×2 的 PCIe 端口，其中每一个 PCIe 端口都可以挂接 EP。除此之外 PCIe 总线还可以使用 PCIe 桥，将 PCIe 总线转换为 PCI 总线或者 PCI-X 总线，之后挂接 PCI/PCI-X 设备。多数 x86 处理器系统使用这种结构连接 PCIe 或者 PCI 设备。

采用这种结构，有利于处理器系统对外部设备进行统一管理，因为所有外部设备都属于同一个 PCI 总线域，系统软件可以使用 PCI 总线协议统一管理所有外部设备。然而这种外部设备管理方法并非尽善尽美，使用这种结构时，需要注意存储器控制器使用的寄存器也被映射为 PCI 总线空间，从而属于 PCI 总线域，而主存储器（如 DDR 内存空间）仍然属于存储器域。因此在这种结构中，存储器域与 PCI 总线域的划分并不明晰。

在 PCIe 总线规范中并没有明确提及 PCIe 主桥，而使用 RC 概括除了处理器之外的所有与 PCIe 总线相关的内容。在 PCIe 体系结构中，RC 是一个很模糊也很混乱的概念。Intel 使用 PCI 总线的概念管理所有外部设备，包括与这些外部设备相关的寄存器，因此在 RC 中包含一些实际上与 PCIe 总线无关的寄存器。使用这种方式有利于系统软件使用相同的平台管理所有外部设备，也利于平台软件的一致性，但是仍有其不足之处。

PCIe 总线在 x86 处理器中始终处于核心地位。Intel 也借 PCIe 总线统一管理所有外部设备，并以此构建基于 PCIe 总线的 PC 生态系统（Ecosystem）。PCI/PCIe 总线在 x86 处理器系统中的地位超乎想象，而且并不仅局限于硬件层面。

2. PowerPC 处理器

PowerPC 处理器挂接外部设备使用的拓扑结构与 x86 处理器不同。在 PowerPC 处理器中，虽然也含有 PCI/PCIe 总线，但是仍然有许多外部设备并不是连接在 PCI 总线上的。在 PowerPC 处理器中，PCI/PCIe 总线并没有在 x86 处理器中的地位。在 PowerPC 处理器中，还含有许多内部设备，如 TSEC（Three Speed Ethernet Controller）和一些内部集成的快速设备，与 SoC 平台总线直接相连，而不与 PCI/PCIe 总线相连。在 PowerPC 处理器中，PCI/PCIe 总线控制器连接在 SoC 平台总线的下方。

Freescale 即将发布的 P4080 处理器，采用的互连结构与之前的 PowerPC 处理器有较大的不同。P4080 处理器是 Freescale 第一颗基于 E500mc 内核的处理器。E500mc 内核与之前的 E500 V2 和 V1 相比，从指令流水线结构、内存管理和中断处理上说并没有本质的不同。E500mc 内核内置了一个 128 KB 大小的 L2 Cache，该 Cache 连接在 BSB 总线上；而 E500 V1/V2 内核中并不含有 L2 Cache，而仅含有 L1 Cache，而且与 FSB 直接相连。在 E500mc 内核中，还引入了虚拟化的概念。

P4080 处理器共集成了 8 个 E500mc 内核，并使用 CoreNet 连接这 8 个 E500mc 内核，由 CoreNet 互连的处理器使用交换结构进行数据交换，而不是基于共享总线结构。在 P4080 处理器中，一些快速外部设备，如 DDR 控制器、以太网控制器和 PCI/PCIe 总线接口控制器也是直接或者间接地连接到 CoreNet 中，L3 Cache 也是连接到 CoreNet 中。P4080 处理器的拓扑结构如图 4-8 所示。

目前 Freescale 并没有公开 P4080 处理器的 L1、L2 和 L3 Cache 如何进行 Cache 共享一致性。多数采用 CoreNet 架构互连的处理器系统使用目录表法进行 Cache 共享一致性。但是 P4080 处理器并不是一个追求峰值运算的 SMP 处理器系统，而针对 Data Plane 的应用，因此 P4080 处理器可能并没有使用基于目录表的 Cache 一致性协议。在基于全互连网络的处理器系统中如果使用“类总线监听法”进行 Cache 共享一致性，将不利于多个 CPU 共享同一个

存储器系统，在 Cache 一致性的处理过程中容易形成瓶颈。

如图 4-8 所示，P4080 处理器的设计重点并不是 E500mc 内核，而是 CoreNet。CoreNet 内部由全互连网络组成，其中任意两个端口间的通信并不会影响其他端口间的通信。与 MPC8548 处理器相同，P4080 处理器也使用 OCeaN^①结构连接 PCIe 与 RapidIO 接口。

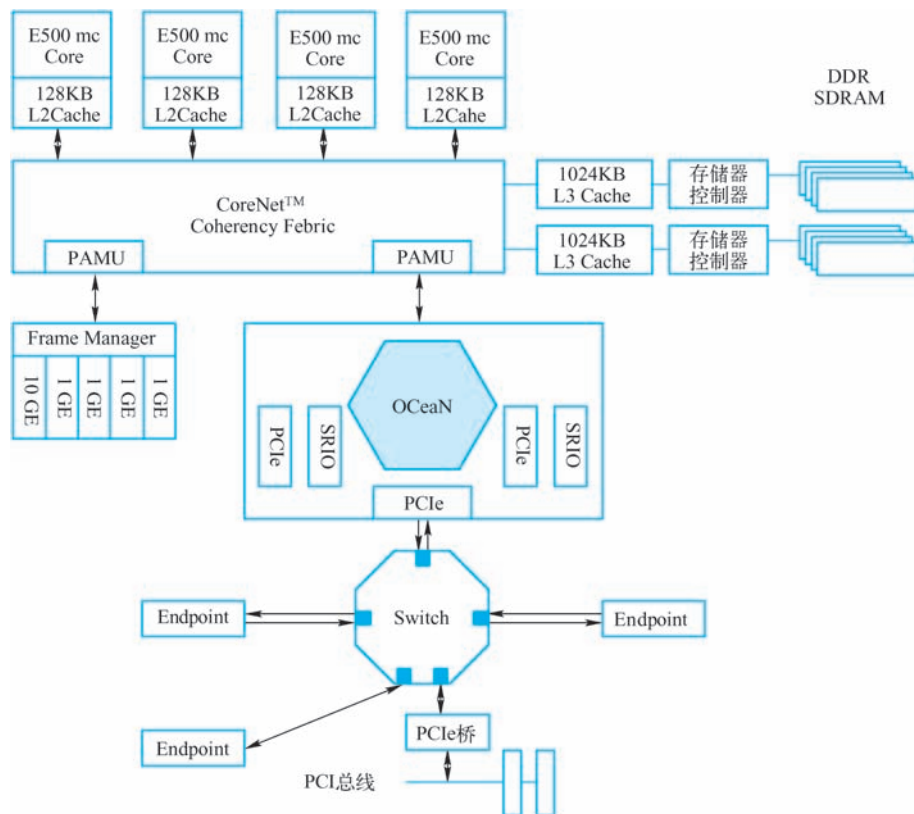


图 4-8 基于 PCIe 总线的 PowerPC 处理器系统

在 P4080 处理器中不存在 RC 的概念，仅存在 PCIe 总线控制器，当然也可以认为在 P4080 处理器中，PCIe 总线控制器即为 RC。P4080 处理器内部含有 3 个 PCIe 总线控制器，如果该处理器需要连接更多的 PCIe 设备时，需要使用 Switch 扩展 PCIe 链路。

在 P4080 处理器中，所有外部设备与处理器内核都连接在 CoreNet 中，而不使用传统的 SoC 平台总线^②进行连接，从而在有效提高了处理器与外部设备间通信带宽的同时，极大降低了访问延时。此外 P4080 处理器系统使用 PAMU（Peripheral Access Management Unit）分隔外设地址空间与 CoreNet 地址空间。在这种结构下，10 GE/1 GE 接口使用的地址空间与 PCI 总线空间独立。

P4080 处理器使用的 PAMU 是对 MPC8548 处理器 ATMU 的进一步升级。使用这种结构时，外部设备使用的地址空间、PCI 总线域地址空间和存储器域地址空间的划分更加明晰。

① OCeaN 是一个基于交叉矩阵的总线结构，连接在 OCeaN 中的外部设备可以直接通信，而不相互干扰。

② 这种方式也可以被认为是 SoC 平台总线从共享总线结构升级到 Switch 结构。

在 P4080 处理器中, 存储器控制器和存储器都属于一个地址空间, 即存储器域地址空间。此外这种结构还使用 OCeaN 连接 SRIO[⊖]和 PCIe 总线控制器, 使得在 OCeaN 中的 PCIe 端口之间[⊖]可以直接通信, 而不需要通过 CoreNet, 从而减轻了 CoreNet 的负载。

从内核互连和外部设备互连的结构上看, 这种结构具有较大的优势。但是采用这种结构需要使用占用芯片更多的资源, CoreNet 的设计也十分复杂。而最具挑战的问题是, 在这种结构之下, Cache 共享一致性模型的设计与实现。

在 Boxboro EX 处理器系统中, 可以使用 QPI 将多个处理器系统进行点到点连接, 也可以组成一个全互连的处理器系统。这种结构与 P4080 处理器使用的结构类似, 但是 Boxboro EX 处理器系统包含的 CPU 更多。

这种全互连的处理器结构也许是未来多核处理器发展的趋势, 但是在没有合理地解决多核处理器可编程性问题之前, 这种结构很可能不会被系统软件高效地利用, 这也是这一结构所面临的挑战。

3. 基于 PCIe 总线的通用处理器结构

在不同的处理器系统中, RC 的实现有较大差异。PCIe 总线规范并没有规定 RC 的实现细则。在有些处理器系统中, RC 相当于 PCIe 主桥, 也有的处理器系统也将 PCIe 主桥称为 PCIe 总线控制器。而在 x86 处理器系统中, RC 除了包含 PCIe 总线控制器之外, 还包含一些其他组成部件, 因此 RC 并不等同于 PCIe 总线控制器。

如果一个 RC 可以提供多个 PCIe 端口, 这种 RC 也被称为多端口 RC。如 MPC8572 处理器的 RC 可以直接提供 3 条 PCIe 链路, 因此可以直接连接 3 个 EP。如果 MPC8572 处理器需要连接更多 EP 时, 需要使用 Switch 进行链路扩展。

而在 x86 处理器系统中, RC 并不是存在于一个芯片中, 如在 Montevina 平台中, RC 由 MCH 和 ICH 两个芯片组成。有关 Montevina 平台的详细说明见第 5 章。本节并不对 x86 和 PowerPC 处理器使用的 PCIe 总线结构做进一步讨论, 而只介绍这两种结构的相同之处。一个通用的、基于 PCIe 总线的处理器系统如图 4-9 所示。

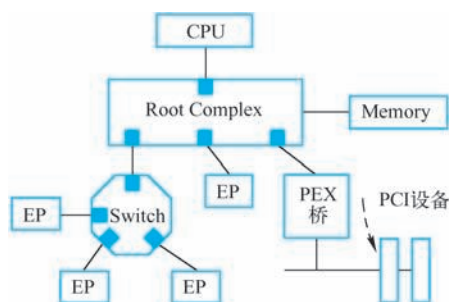


图 4-9 基于 PCIe 总线的通用处理器系统

图中所示的结构将 PCIe 总线端口、存储器控制器等一系列与外部设备有关的接口都集成在一起, 并统称为 RC。RC 具有一个或者多个 PCIe 端口, 可以连接各类 PCIe 设备。PCIe

⊖ SRIO 为串行 RapidIO。

⊖ PCIe 端口之间的直接通信过程也称为 Peer-to-Peer 传送方式。

设备包括 EP（如网卡、显卡等设备）、Switch 和 PEX 桥。

PCIe 总线采用端到端的连接方式，每一个 PCIe 端口只能连接一个 EP，当然 PCIe 端口也可以连接 Switch 进行链路扩展。通过 Switch 扩展出的 PCIe 链路可以继续挂接 EP 或者其他 Switch。

4.2.2 RC 的组成结构

RC 是 PCIe 体系结构的一个重要组成部件，也是一个较为混乱的概念。RC 的提出与 x86 处理器系统密切相关。事实上，只有 x86 处理器才存在 PCIe 总线规范定义的“标准 RC”，而在多数处理器系统中，并不含有在 PCIe 总线规范中涉及到的与 RC 相关的全部概念。

不同处理器系统的 RC 设计并不相同，在图 4-7 中的处理器系统中，RC 包括存储器控制器、两个 FSB-to-PCIe 桥。而在图 4-8 中的 PowerPC 处理器系统中，RC 的概念并不明晰。在 PowerPC 处理器中并不存在真正意义上的 RC，而仅包含 PCIe 总线控制器。

在 x86 处理器系统中，RC 内部集成了一些 PCI 设备、RCRB（RC Register Block）和 Event Collector 等组成部件。其中 RCRB 由一系列“管理存储器系统”的寄存器组成，而仅存在于 x86 处理器中；而 Event Collector 用来处理来自 PCIe 设备的错误消息报文和 PME 消息报文。RCRB 寄存器组属于 PCI 总线域地址空间，x86 处理器访问 RCRB 的方法与访问 PCI 设备的配置寄存器相同。在有些 x86 处理器系统中，RCRB 在 PCI 总线 0 的设备 0 中。

RCRB 是 x86 处理器所独有的，PowerPC 处理器也含有一组“管理存储器系统”的寄存器，这组寄存器与 RCRB 所实现的功能类似。但是在 PowerPC 处理器中，该组寄存器以 CCSRBAR 寄存器为基地址，处理器采用存储器映像方式访问这组寄存器。

如果将 RC 中的 RCRB、内置的 PCI 设备和 Event Collector 去除，该 RC 的主要功能与 PCI 总线中的 HOST 主桥类似，其主要作用是完成存储器域到 PCI 总线域的地址转换。但是随着虚拟化技术的引入，尤其是引入 MR-IOV 技术之后，RC 的实现变得异常复杂。有关 MR-IOV 技术的详细说明见第 13.3.2 节。

但是 RC 与 HOST 主桥并不相同，RC 除了完成地址空间的转换之外，还需要完成物理信号的转换。在 PowerPC 处理器的 RC 中，来自 OCeAN 或者 FSB 的信号协议与 PCIe 总线信号使用的电气特性并不兼容，使用的总线事务也并不相同，因此必须进行信号协议和总线事务的转换。

在 P4080 处理器中，RC 的下游端口可以挂接 Switch 扩展更多的 PCIe 端口，也可以只挂接一个 EP。在 P4080 处理器的 RC 中，设置了一组 Inbound 和 Outbound 寄存器组，用于存储器域与 PCI 总线域之间地址空间的转换；而 P4080 处理器的 RC 还可以使用 Outbound 寄存器组将 PCI 设备的配置空间直接映射到存储器域中。PowerPC 处理器在处理 PCI/PCIe 接口时，都使用这组 Inbound 和 Outbound 寄存器组。

在 P4080 处理器中，RC 可以使用 PEX_CONFIG_ADDR 与 PEX_CONFIG_DATA 寄存器对 EP 进行配置读写，这两个寄存器与 MPC8548 处理器 HOST 主桥的 PCI_CONFIG_ADDR 和 PCI_CONFIG_DATA 寄存器类似，本章不再详细介绍这组寄存器。

而 x86 处理器的 RC 设计与 PowerPC 处理器有较大的不同，实际上和大多数处理器系统都不相同。x86 处理器赋予了 RC 新的含义，PCIe 总线规范中涉及的 RC 也以 x86 处理器为例进行说明，而且一些在 PCIe 总线规范中出现的最新功能也在 Intel 的 x86 处理器系统中率

先实现。在 x86 处理器系统中的 RC 实现也比其他处理器系统复杂得多。深入理解 x86 处理器系统的 RC 对于理解 PCIe 体系结构非常重要，因此本书将以 Montivina 平台为例详细介绍 x86 处理器中的 RC，其详细描述见第 5 章。

4.2.3 Switch

第 4.1.4 节简单介绍了在 PCIe 总线中，如何使用 Switch 进行链路扩展，本节主要介绍 Switch^①的内部结构。从系统软件的角度上看，每一个 PCIe 链路都占用一个 PCI 总线号，但是一条 PCIe 链路只能连接一个 PCI 设备、Switch、EP 或者 PCIe 桥片。PCIe 总线使用端端的连接方式，一条 PCIe 链路只能连接一个设备。

一个 PCIe 链路需要挂接多个 EP 时，需要使用 Switch 进行链路扩展。一个标准 Switch 具有一个上游端口和多个下游端口。上游端口与 RC 或者其他 Switch 的下游端口相连，而下游端口可以与 EP、PCIe-to-PCI-X/PCI 桥或者下游 Switch 的上游端口相连。

PCIe 总线规范还支持一种特殊的连接方式，即 Crosslink 连接方式。使用这种方式时，Switch 的下游端口可以与其他 Switch 的下游端口直接连接，上游端口也可以其他 Switch 的上游端口直接连接。在 PCIe 总线规范中，Crosslink 连接方式是可选的，并不要求 PCIe 设备一定支持这种连接方式。

在 PCIe 体系结构中，Switch 的设计难度仅次于 RC，Switch 也是 PCIe 体系结构的核心所在。而从系统软件的角度上看，Switch 内部由多个 PCI-to-PCI 桥组成，其中每一个上游和下游端口都对应一个虚拟 PCI 桥。在一个 Switch 中有多少个端口，在其内部就有多少个虚拟 PCI 桥，就有多少个 PCI 桥配置空间。值得注意的是，在 Switch 内部还具有一条虚拟的 PCI 总线，用于连接各个虚拟 PCI 桥，系统软件在初始化 Switch 时，需要为这条虚拟 PCI 总线编号。Switch 的等效逻辑图如图 4-10 所示。

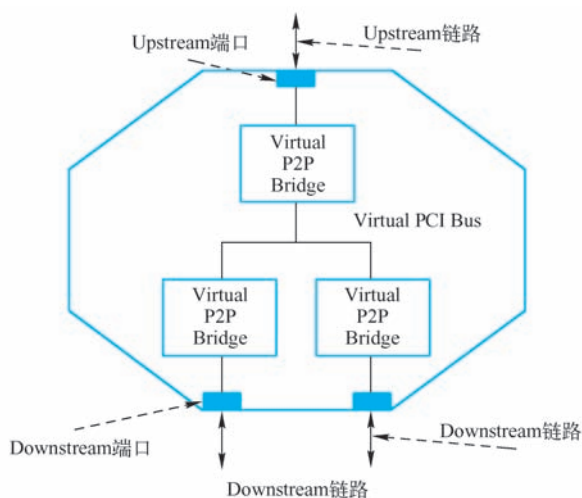


图 4-10 Switch 的等效逻辑图

① PCIe 总线中的 Switch 与网络应用的 Switch 的功能并不相同，而与网络应用中的 Route 功能接近。

Switch 需要处理 PCIe 总线传输过程中的 QoS 问题^①。PCIe 总线的 QoS 要求 PCIe 总线区别对待优先权不同的数据报文，而且无论 PCIe 总线的某一个链路多么拥塞，优先级高的报文，如等时报文（Isochronous Packet）都可以获得额定的数据带宽。而且 PCIe 总线需要保证优先级较高的报文优先到达。PCIe 总线采用虚拟多通路 VC 技术^②，并在这些数据报文中设定一个 TC（Traffic Class）标签，该标签由 3 位组成，将数据报文根据优先权分为 8 类，这 8 类数据报文可以根据需要选择不同的 VC 进行传递。

在 PCIe 总线中，每一条数据链路上最多可以支持 8 个独立的 VC。每个 VC 可以设置独立的缓冲，用来接收和发送数据报文。在 PCIe 体系结构中，TC 和 VC 紧密相连，TC 与 VC 之间的关系是“多对一”。

TC 可以由软件设置，系统软件可以选择某类 TC 由哪个 VC 进行传递。其中一个 VC 可以传递 TC 不相同的数据报文，而 TC 相同的数据报文在指定一个 VC 传递之后，不能再使用其他 VC。在许多处理器系统中，Switch 和 RC 仅支持一个 VC，而 x86 处理器系统和 PLX 的 Switch 中可以支持两个 VC。

下文将以一个简单的例子说明如何使用 TC 标签和多个 VC，以保证数据传送的服务质量。将 PCIe 总线的端到端数据传递过程模拟为使用汽车将一批货物从 A 点运送到 B 点。如果不考虑服务质量，可以使用一辆汽车运送所有这些货物，经过多次往返就可以将所有货物从 A 点运到 B 点。但是这样做会耽误一些需要在指定时间内到达 B 点的货物。有些货物，如一些急救物资、EMS 等其他优先级别较高的货物，必须及时地从 A 点运送到 B 点。这些急救物资的运送应该有别于其他普通物资的运送。

为此首先将不同种类的货物进行分类，将急救物资定义为 TC3 类货物，EMS 定义为 TC2 类货物，平信定义为 TC1 类货物，一般包裹定义为 TC0 类货物，最多可以提供 8 种 TC 类标签进行货物分类。

之后使用 8 辆汽车，分别是 VC0~VC7 运送这些货物，其中 VC7 的速度最快，而 VC0 的速度最慢。当发生堵车事件时，VC7 优先行驶，VC0 最后行驶。然后我们使用 VC3 运送急救物资，VC2 运送 EMS，VC1 运送平信，VC0 运送包裹，当然使用 VC0 同时运送平信和包裹也是可以的，但是平信或者包裹不能使用一种以上的汽车运送，如平信如果使用了 VC1 运输，就不能使用 VC0。因为 TC 与 VC 的对应关系是“多对一”的关系。

采用这种分类运输的方法，可以做到在 A 点到 B 点带宽有限的情况下，仍然可以保证急救物资和 EMS 可以及时到达 B 点，从而提高了服务质量。

PCIe 总线除了解决数据传送的 QoS 问题之外，还进一步考虑如何在链路传递过程中，使用流量控制机制防止拥塞。PCIe 总线的流量控制机制较为复杂，第 9 章将介绍和流量控制相关的内容。

在 PCIe 体系结构中，Switch 处于核心地位。PCIe 总线使用 Switch 进行链路扩展，在 Switch 中，每一个端口对应一个虚拟 PCI 桥。深入理解 PCI 桥是理解 Switch 软件组成结构的基础。目前 PCIe 总线提出了 MRA-Switch 的概念，这种 Switch 与传统 Switch 有较大的区别，

① 在 PCIe 体系结构中，RC 和 EP 也需要处理 QoS。

② 有关多通路 VC 的详细说明见第 9 章。

有关这部分内容详见第 13.3 节。

4.2.4 VC 和端口仲裁

在 Switch 中存在多个端口，其中来自不同 Ingress 端口的报文可以发向同一个 Egress 端口，因此 Switch 必须要解决端口仲裁和路由选径的问题。所谓端口仲裁指来自不同 Ingress 端口的报文到达同一个 Egress 端口的报文通过顺序，端口仲裁机制如图 4-11 所示。

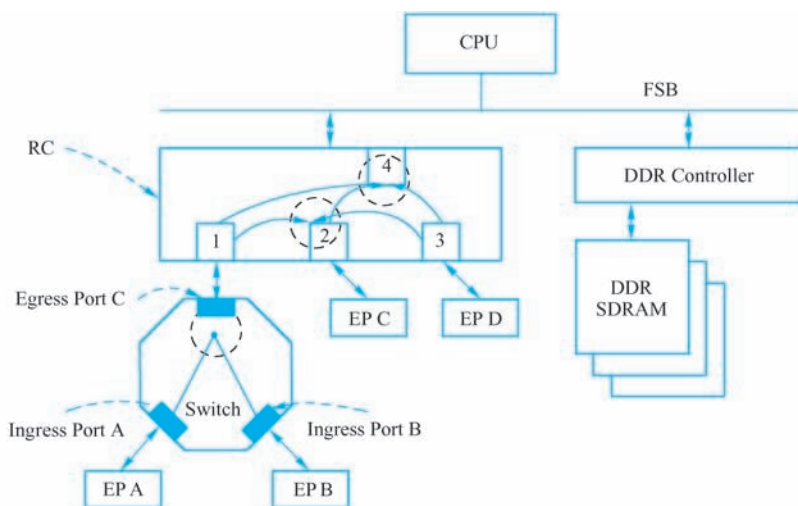


图 4-11 PCIe 总线基于端口的仲裁机制

在一个 Switch 中设有仲裁器，该仲裁器规定了数据报文通过 Switch 的规则。在 PCIe 总线中存在两种仲裁机制，分别是基于 VC 和基于端口的仲裁机制。端口仲裁机制主要针对 RC 和 Switch，当多个 Ingress 端口需要向同一个 Egress 端口发送数据报文时需要进行端口仲裁。具体地讲，在 PCIe 体系结构中有三个端口，需要进行端口仲裁。

- Switch 的 Egress 端口。当 EP A 和 EP B 同时访问 EP C、D 或者 DDR-SDRAM 时，需要通过 Switch 的 Egress 端口 C。此时 Switch 需要进行端口仲裁确定是 EP A 的数据报文还是 EP B 的数据报文优先通过 Egress 端口 C。
- 多端口 RC 的 Egress 端口。当 RC 的端口 1 和端口 3 同时访问 Endpoint C 时，RC 的端口 2 需要进行端口仲裁，决定来自 RC 哪个端口的数据可以率先通过。
- RC 通往主存储器的端口。当 RC 的端口 1、端口 2 和端口 3 同时访问 DDR 控制器时，这些数据报文将通过 RC 的 Egress 端口 4，此时需要进行端口仲裁。

在 PCIe 体系结构中，链路的端口仲裁需要根据每一个 VC 独立设置，而且可以使用不同的算法进行端口仲裁。

下文以图 4-11 中 Switch 的两个 Ingress 端口 A 和 B 向 Egress 端口 C 发送数据报文为例，简要说明端口仲裁和 VC 仲裁的使用方法，其过程如图 4-12 所示。

基于 VC 的仲裁是指发向同一个端口的数据报文，根据使用的 VC 而进行仲裁的方式。当来自端口 B 和端口 A 数据报文（分别使用 VC0 和 VC1 通路）在到达端口 C 之前，需要首先进行端口仲裁后，才能进行 VC 仲裁。PCIe 总线规定了 3 种 VC 仲裁方式，分别为 Strict Priority，RR（Round Robin）和 WRR（Weighted Round Robin）算法。

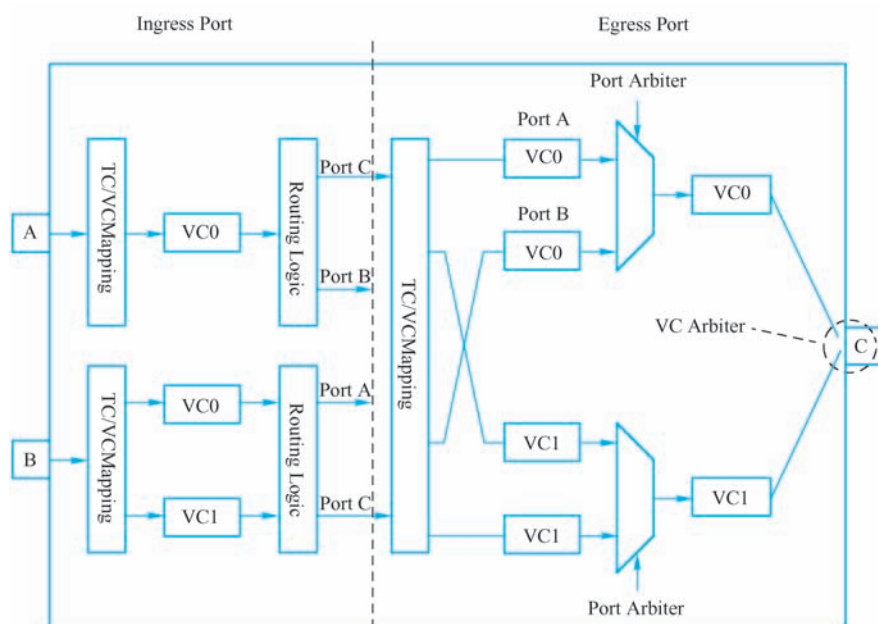


图 4-12 VC 仲裁示意图

当使用 Strict Priority 仲裁方式时，发向 VC7 的数据报文具有最高的优先级，而发向 VC0 的数据报文优先级最低。PCIe 总线允许对 Switch 或者 RC 的部分 VC 采用 Strict Priority 方式进行仲裁，而对其他 VC 采用 RR 和 WRR 算法，如 VC7~VC4 采用 Strict Priority 方式，而采用其他方式处理 VC3~VC0。

使用 RR 方式时，所有 VC 具有相同的优先级，所有 VC 轮流使用 PCIe 链路。WRR 方式与 RR 算法类似，但是可以对每一个 VC 进行加权处理，采用这种方式可以适当提高 VC7 的优先权，而将 VC0 的优先权适当降低。

我们假定 Ingress 端口 A 和 Ingress B 向 Egress 端口 C 进行数据传递时，使用两个 VC 通路，分别是 VC0 和 VC1。其中标签为 TC0~TC3 的数据报文使用 VC0 传送，而标签为 TC4~TC7 数据报文使用 VC1 传送。

而数据报文在离开 Egress 端口 C 时，需要首先进行端口仲裁，之后再通过 VC 仲裁，决定哪个报文优先传送。数据报文从 Ingress A/B 端口发送到 Egress C 端口时，将按照以下步骤进行处理。

1) 首先到达 Ingress A/B 端口的数据报文，将根据该端口的 TC/VC 映射表^①决定使用该端口的哪个 VC 通道。如图 4-12 所示，假设发向端口 A 的数据报文使用 TC0~TC3，而发向端口 B 的数据报文使用 TC0~TC7，这些数据报文在端口 A 中仅使用了 VC0 通道，而在端口 B 中使用了 VC0 和 VC1 两个通道。

2) 数据报文在端口中传递时，将通过路由部件（Routing Logic），将报文发送到合适的端口。如图 4-12 所示，端口 C 可以接收来自端口 A 或端口 B 的数据报文。

① 该表存在于 PCI Express Extended Capabilities 结构中，详见第 4.3.3 小节。

3) 当数据报文到达端口 C 时, 首先需要经过 TC/VC 映射表, 确定在端口 C 中使用哪个 VC 通路接收不同类型的数据报文。

4) 对于端口 C, 其 VC0 通道可能会被来自端口 A 的数据报文使用, 也可能被来自端口 B 的数据报文使用。因此在 PCIe 的 Switch 中必须设置一个端口仲裁器, 决定来自不同数据端口的数据报文如何使用 VC 通路。

5) 数据报文通过端口仲裁后, 获得 VC 通路的使用权之后, 还需要经过 Switch 中的 VC 仲裁器, 将数据报文发送到实际的物理链路中。

PCIe 总线规定, 系统设计者可以使用以下三种方式进行端口仲裁。

1) Hardware-fixed 仲裁策略。如在系统设计时, 采用固化的 RR 仲裁方法。这种方法的硬件实现原理较为简单, 此时系统软件不能对端口仲裁器进行配置。

2) WRR 仲裁策略, 即加权的 RR 仲裁策略, 该算法和 Time-Based WRR 算法的描述见第 4.3.3 小节。

3) Time-Based WRR 仲裁策略, 基于时间片的 WRR 仲裁策略, PCIe 总线可以将一个时间段分为若干个时间片 (Phase), 每个端口占用其中的一个时间片, 并根据端口使用这些时间片的多少对端口进行加权的一种方法。使用 WRR 和 Time-Based WRR 仲裁策略, 可以在某种程度上提高 PCIe 总线的 QoS。

PCIe 设备的 Capability 寄存器规定了端口仲裁使用的算法, 详见第 4.3.3 小节。有些 PCIe 设备并没有提供多种端口仲裁算法, 可能也并不含有 Capability 寄存器。此时该 PCIe 设备使用 Hardware-fixed 仲裁策略。

4.2.5 PCIe-to-PCI/PCI-X 桥片

本书将 PCIe-to-PCI/PCI-X 桥片简称为 PCIe 桥片。该桥片有两个作用。

- 将 PCIe 总线转换为 PCI 总线, 以连接 PCI 设备。在一个没有提供 PCI 总线接口的处理器中, 需要使用这类桥片连接 PCI 总线的外设。许多 PowerPC 处理器在提供 PCIe 总线的同时, 也提供了 PCI 总线, 因此 PCIe-to-PCI 桥片对基于 PowerPC 处理器系统并不是必须的。
- 将 PCI 总线转换为 PCIe 总线 (这也被称为 Reverse Bridge), 连接 PCIe 设备。一些低端的处理器并没有提供 PCIe 总线, 此时需要使用 PCIe 桥将 PCI 总线转换为 PCIe 总线, 才能与其他 PCIe 设备互连。这种用法初看比较奇怪, 但是在实际应用中, 确实有使用这一功能的可能。本节主要讲解 PCIe 桥的第一个作用。

PCIe 桥的一端与 PCIe 总线相连, 而另一端可以与一条或者多条 PCI 总线连接。其中可以连接多个 PCI 总线的 PCIe 桥也被称为多端口 PCIe 桥。

PCIe 总线规范提供了两种多端口 PCIe 桥片的扩展方法。多端口 PCIe 桥片指具有一个上游端口和多个下游端口的桥片。其中上游端口连接 PCIe 链路, 而下游端口推出 PCI 总线, 连接 PCI 设备。这种桥片的结构如图 4-13 所示。

PCIe 总线规范并没有强制厂商实现多端口 PCIe 桥的办法。但是值得注意的是, 使用右图扩展多条 PCI 总线时, 在多端口 PCIe 桥中包含一个虚拟的 PCI 总线, 即 Bus 2。系统软件对 PCI 总线进行深度优先搜索 DFS (Depth-First Search) 时, 对图 4-13 左图和右图的处理有些区别。目前多端口 PCIe 桥多使用右图进行端口扩展。

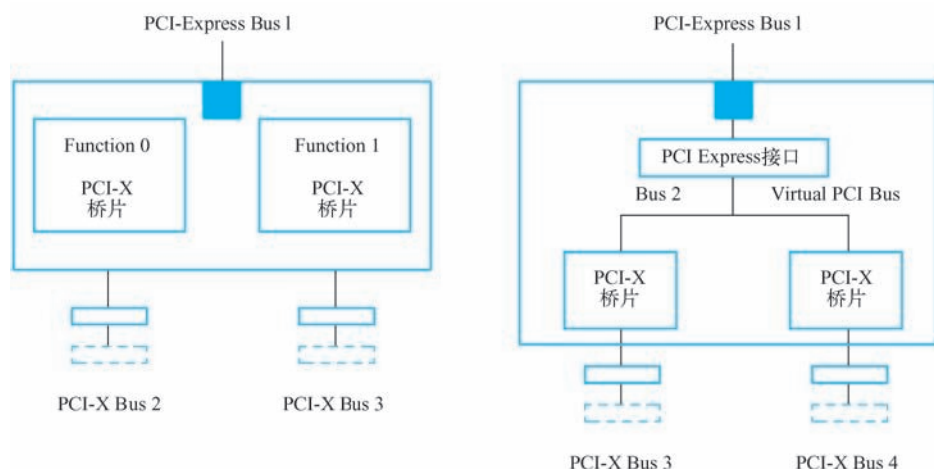


图 4-13 多端口 PCIe 桥的扩展方法

目前虽然 PCIe 总线非常普及，但是仍然有许多基于 PCI 总线的设计，这些基于 PCI 总线的设计可以通过 PCIe 桥，方便地接入到 PCIe 体系结构中。目前有多家半导体厂商可以提供 PCIe 桥片，如 PLX、NXP、Tundra 和 Intel。就功能的完善和性能而言，Intel 的 PCIe 桥无疑是最佳选择，而 PLX 和 Tundra 的 PCIe 桥在嵌入式系统中得到了广泛的应用。

4.3 PCIe 设备的扩展配置空间

本书在第 2.3.2 小节讲述了 PCI 设备使用的基本配置空间。这个基本配置空间共由 64 个字节组成，其地址范围为 0x00~0x3F，这 64 个字节是所有 PCI 设备必须支持的。事实上，许多 PCI 设备也仅支持这 64 个配置寄存器。

此外 PCI/PCI-X 和 PCIe 设备还扩展了 0x40~0xFF 这段配置空间，在这段空间主要存放一些与 MSI 或者 MSI-X 中断机制和电源管理相关的 Capability 结构。其中所有能够提交中断请求的 PCIe 设备，必须支持 MSI 或者 MSI-X Capability 结构。

PCIe 设备还支持 0x100~0xFFF 这段扩展配置空间。PCIe 设备使用的扩展配置空间最大为 4KB，在 PCIe 总线的扩展配置空间中，存放 PCIe 设备所独有的一些 Capability 结构，而 PCI 设备不能使用这段空间。

在 x86 处理器中，使用 CONFIG_ADDRESS 寄存器与 CONFIG_DATA 寄存器访问 PCIe 配置空间的 0x00~0xFF，而使用 ECAM 方式访问 0x000~0xFFF 这段空间；而在 PowerPC 处理器中，可以使用 CFG_ADDR 和 CFG_DATA 寄存器访问 0x000~0xFFF，详见第 2.2 节。

PCI-X 和 PCIe 总线规范要求其设备必须支持 Capabilities 结构。在 PCI 总线的基本配置空间中，包含一个 Capabilities Pointer 寄存器，该寄存器存放 Capabilities 结构链表的头指针。在一个 PCIe 设备中，可能含有多个 Capability 结构，这些寄存器组成一个链表，其结构如图 4-14 所示。

其中每一个 Capability 结构都有唯一的 ID 号，每一个 Capability 寄存器都有一个指针，这个指针指向下一个 Capability 结构，从而组成一个单向链表结构，这个链表的最后一个

Capability 结构的指针为 0。

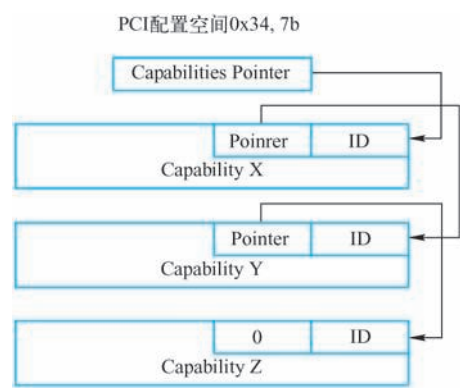


图 4-14 PCIe 总线 Capability 结构的组成

一个 PCIe 设备可以包含多个 Capability 结构，包括与电源管理相关、与 PCIe 总线相关的结构、与中断请求相关的 Capability 结构、PCIe Capability 结构和 PCIe 扩展的 Capability 结构。在本书的其他章节也将讲述这些 Capability 结构，读者在继续其他章节的学习之前，需要简单了解这些 Capability 结构的寄存器组成和使用方法。

其中读者需要重点关注的是 Power Management 和 MSI/MSI-X Capability 结构，Power Management 结构将在本节介绍，而在第 10 章将详细讨论 MSI/MSI-X Capability 结构。在 PCIe 总线规范中，定义了较多的 Capability 结构，这些结构适用于不同的应用场合，在一个指定的 PCIe 设备中，并不一定支持本书涉及的所有 Capability 结构。系统软件程序员也不需要完全掌握 PCIe 总线规范定义的这些 Capability 结构。

4.3.1 Power Management Capability 结构

PCIe 总线使用的软件电源管理机制与 PCI PM（Power Management）兼容。而 PCI 总线的电源管理机制需要使用 Power Management Capability 结构，该结构由一些和 PCI/PCI-X 和 PCIe 总线的电源管理相关的寄存器组成，包括 PMCR（Power Management Capabilities Register）和 PMCSR（Power Management Control and Status Register），其结构如图 4-15 所示。

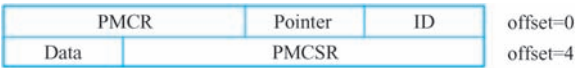


图 4-15 Power Management Capability 结构

Capability ID 字段记载 Power Management Capability 结构的 ID 号，其值为 0x01。在 PCIe 设备中，每一个 Capability 都有唯一的一个 ID 号，而 Next Capability Pointer 字段存放下一个 Capability 结构的地址。

1. PMCR

PMCR 由 16 位组成，其中所有位和字段都是只读的。该寄存器的主要目的是记录当前 PCIe 设备的物理属性，系统软件需要从 PMCR 寄存器中获得当前 PCIe 设备的信息后，才能对 PMCSR 进行修改。该寄存器的结构如图 4-16 所示，其中 PMCR 在 Power Management Capability 结构的第 3~2 字节中。

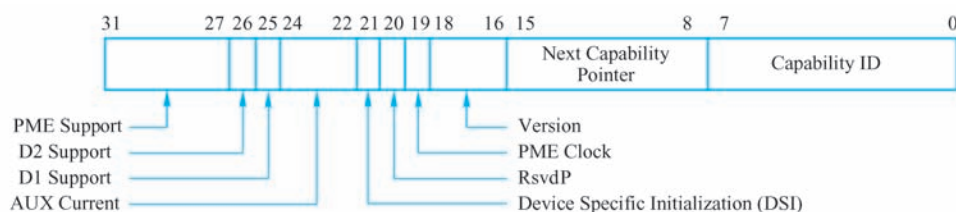


图 4-16 PMCR

- Version 字段只读，记录 Power Management Capability 结构的版本号。
- PME Clock 位只读，该位没有被 PCIe 总线使用[⊖]，硬件逻辑必须将其接为 0。PCI 设备可以使用 PME# 信号通知设备改变电源状态，在 PCI 总线中，如果 PME# 信号需要使用时钟（PCI Clock）时，该位为 1；否则该位为 0。PCI 设备改变电源状态时，将 PME# 信号置为有效，向处理器系统提交请求。系统软件将这个请求处理完毕后，将通知这个 PCI 设备，之后该 PCI 设备将 PME# 信号置为无效。
- RsvdP 字段为系统保留字段。
- DSI（Device Specific Initialization）位只读。某些 PCIe 设备在上电时处于某种工作模式，之后可以通过重新配置运行在其他工作模式中，此时该设备需要使用 DSI 位表示该设备可以使用自定义的电源工作方式。
- AUX（Auxiliary device）Current 字段只读，表示 PCIe 设备需要使用辅助电源的电流强度。PCIe 设备需要使用两种电源，一个是主电源 V_{cc} ，另一个是辅助电源 V_{aux} 。当 PCIe 设备进入某种节能状态时，主电源将停止供电，而辅助电源需要继续供电。该字段记录 V_{aux} 使用的电流强度，其最大值为 375 mA，最小值为 0，即不使用 V_{aux} 。
- D2 和 D1 位只读。D2 位为 1 表示 PCIe 设备支持 D2 状态；D1 位为 1 表示 PCIe 设备支持 D1 状态。PCI PM 机制规定 PCIe 设备可以支持四种状态，分别为 D0~D3 状态。PCIe 设备处于 D0 状态时的功耗最高，处于 D3 状态时最低。多数支持电源管理的 PCIe 设备仅支持 D0 状态和 D3 状态，而 D1 和 D2 状态可选，有关这四种状态的详细说明见第 8.4.1 节。
- PME Support 字段只读，存放 PCIe 设备支持的电源状态。第 27 位为 1 时，表示 PCIe 设备处于 D0 状态时，可以发送 PME 消息；第 28 位为 1 时，表示 PCIe 设备处于 D1 状态时，可以发送 PME 消息；第 29 位为 1 时，表示 PCIe 设备处于 D2 状态时可以发送 PME 消息；第 30 位为 1 时，表示 PCIe 设备处于 D3_{hot} 状态时可以发送 PME 消息；第 31 位为 1 时，表示 PCIe 设备处于 D3_{cold} 状态时可以发送 PME 消息。

2. PMCSR

系统软件可以通过操作 PMCSR，完成 PCIe 设备电源状态的迁移。该寄存器的结构如图 4-17 所示。

⊖ PCIe 总线使用消息报文（PME Message）进行电源管理，PCIe 设备不支持 PME# 信号。

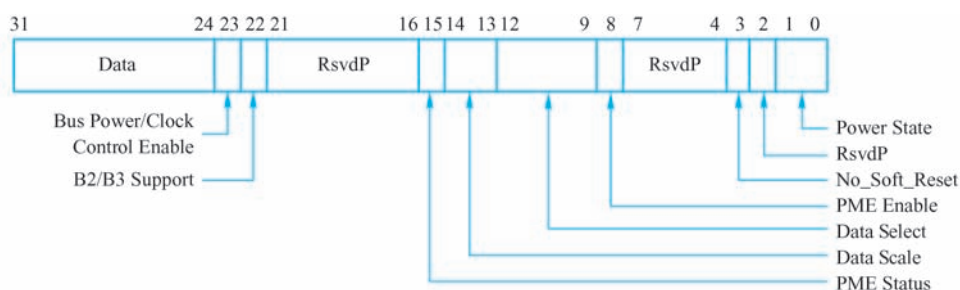


图 4-17 PMCSR

- Power State 字段可读写，该字段记录 PCIe 设备所处的状态。“0b00”与 D0 状态对应；“0b01”与 D1 状态对应；“0b10”与 D2 状态对应；“0b11”与 D3 状态对应。系统软件改变该字段时，PCIe 设备将进行电源状态迁移。
- No_Soft_Reset 位只读。如果该位为 1，PCIe 设备从 D3_{hot} 状态迁移到 D0 状态时，并不需要进行内部复位操作，有关 PCIe 设备配置的现场信息可以由 PCIe 设备的硬件逻辑保存，此时当设备从 D0 状态迁移到 D3_{hot} 状态时，不需要系统软件的干预，其现场由 PCIe 设备主动保存；而该位为 0 时，PCIe 设备从 D3_{hot} 状态迁移到 D0 状态时，需要进行复位操作，因此系统软件在通过改变 Power State 字段使 PCIe 设备从 D0 状态迁移到 D3_{hot} 状态之前，需要保存 PCIe 设备的相关上下文，当 PCIe 设备从 D3_{hot} 状态迁移到 D0 状态时，再进行上下文的恢复操作。
- PME Enable 位，可读写。该位为 1 时，PCIe 设备可以发送 PME 消息；如果为 0，不可以发出 PME 消息。当 PCIe 设备处于 D3_{cold} 状态不能发送 PME 消息时，该位由系统软件设为 0。支持远程唤醒模式的网卡需要将此位使能。
- Data Select 字段可读写，而 Data Scale 字段和 Data 字段只读。系统软件通过这组字段，读取 PCIe 设备处于不同状态时的功耗。首先系统软件置 Data Select 字段为 0~7 之间的数值，其中 0 和 4 与 D0 状态对应，1 和 5 与 D1 状态对应，2 和 6 与 D2 状态对应，3 和 7 与 D3 状态对应；之后系统软件读取 Data Select 和 Data 字段并以此计算在不同状态下 PCIe 设备的功耗。其中 Data Scale 字段记录精度，为 0 时表示该 PCIe 设备不支持这组字段^①；为 1 时表示 Data 字段的数据需要乘以 0.1 后，才能得到 PCIe 设备的功耗，其单位为 W；为 2 时表示 Data 字段的数据需要乘以 0.01；为 3 时表示 Data 字段的数据需要乘以 0.001。
- PME Status 位，该位只读且写 1 清除，对此位写 0 无意义。该位为 1 时表示 PCIe 设备可以正常发送 PME 消息，系统软件对此位写 1 时，将该位清除。该位由硬件逻辑控制，系统软件仅能清除该位，而不能将该位置 1。
- PCIe 总线没有实现 B2/B3 Support 和 Bus Power/Clock Control Enable 位。在 PCI 总线中，Bus Power/Clock Control Enable 位为 1 时使能 PCI 总线的电源和时钟管理，为 0 时表示关闭；当 Bus Power/Clock Control Enable 位为 1 时，B2/B3 Support 位才有意

① 如果 Data 字段为 0，也表示该 PCIe 设备不支持功耗的测量。

义，B2/B3 Support 位为 1 时表示，当 PCI 桥片处于 D3_{hot} 状态时，这个桥片将停止为 Secondary PCI 总线提供时钟；为 0 时表示，当 PCI 桥片处于 D3_{hot} 状态时，将停止为 Secondary PCI 总线提供电源。

4.3.2 PCI Express Capability 结构

PCI Express Capability 结构存放一些和 PCIe 总线相关的信息，包括 PCIe 链路和插槽的信息。有些 PCIe 设备不一定实现了 PCI Express Capability 结构中的所有寄存器，或者并没有提供这些配置寄存器供系统软件访问。

PCI Express Capability 结构的部分寄存器及其相应字段与硬件的具体实现细节相关，本节仅介绍其中一些系统软件程序员需要了解的字段。在该结构中，Cap ID 字段为 PCI Express Capability 结构使用的 ID 号，其值为 0x10。而 Next Capability 字段存放下一个 Capability 寄存器的地址。PCI Express Capability 结构由 PCI Express Capability Register、Device Capabilities、Device Control、Device Status、Link Capabilities、Link Status、Link Control、Slot Capabilities 和 Slot Status 等一系列寄存器组成。本节仅介绍该结构中常用的寄存器。PCI Express Capability 的组成结构如图 4-18 所示。

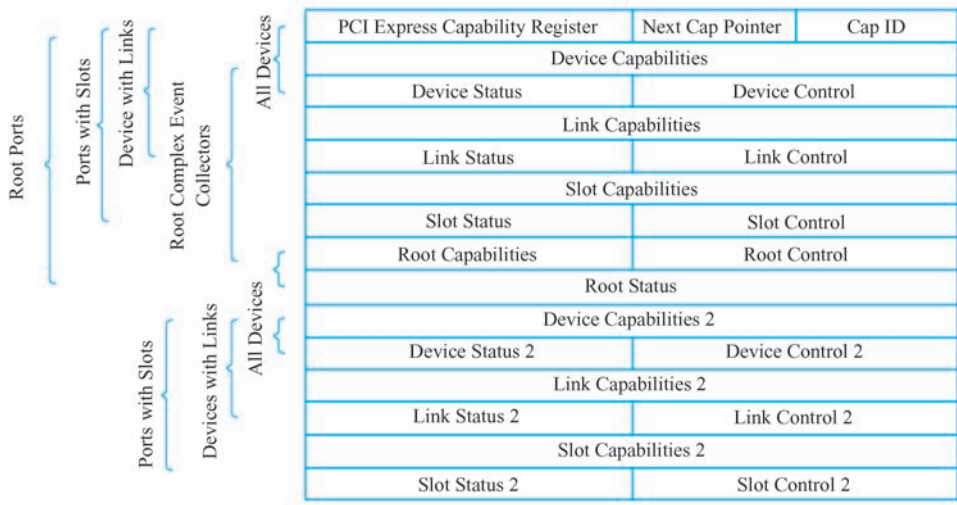


图 4-18 PCI Express Capability 的组成结构

1. PCI Express Capability 寄存器

PCI Express Capability 寄存器存放与 PCIe 设备相关的一些参数，包括版本号信息、端口描述，当前 PCIe 链路是与 PCIe 插槽直接连接还是作为内置的 PCIe 设备等一系列信息。这些参数的详细定义如表 4-3 所示。

表 4-3 PCI Express Capability 寄存器

Bits	定 义	描 述
3:0	Capability Version	存放 PCIe 设备的版本号，如果该设备基于 PCIe 总线规范 2.x，该字段的值为 0x2；如果该设备基于 PCIe 总线规范 1.x，该字段的值为 0x1。该字段只读

(续)

Bits	定 义	描 述
7:4	Device/Port Type	存放 PCIe 设备的属性。0b0000 对应 PCIe 总线的 EP；0b0001 对应 Legacy PCIe 总线的 EP；0b0100 对应 RC 的 Root port；0b0101 对应 Switch 的上游端口；0b0110 对应 Switch 的下游端口；0b0111 对应 PCIe 桥片；0b1000 对应 PCI/PCI-X-to-PCIe 桥片；0b1001 对应 RC 中集成的 EP；0b1010 对应 RC 中的 Event Collector ^① 。该字段只读
8	Slot Implemented	当该位为 1 时，表示和当前端口相连的是一个 PCIe 插槽，而不是 PCIe 设备
13:9	Interrupt Message Number	当 PCI Express Capability 结构的 Slot Status 寄存器或者 Root Status 寄存器的状态发生变化时，该 PCIe 设备可以通过 MSI/MSI-X 中断机制向处理器提交中断请求。该字段存放 MSI/MSI-X 中断机制需要的 Message Data 字段。有关 MSI 中断机制的详细描述见第 10 章

① Event Collector 是 RC 集成的一个功能部件，进行错误检查和处理 PME 消息，该部件可选。

2. Device Capability 寄存器

该寄存器的第 2:0 字段为 “Max_Payload_Size Supported” 字段，该字段存放该设备支持的 Max_Payload_Size 参数的大小，该字段只读，如表 4-4 所示。

表 4-4 PCIe 设备支持的 Max_Payload_Size

Bit [2:0]	支持的 Max_Payload_Size
0b000	128B
0b001	256B
0b010	512B
0b011	1024B
0b100	2048B
0b101	4096B

“Max_Payload_Size Supported” 字段决定了一个 TLP 报文可能使用的最大有效负载，PCIe 总线规定 Max_Payload_Size 参数的最大值为 4096B，但是许多 PCIe 设备并不一定能够支持这么大的有效负载。在实际应用中，一个 PCIe 设备支持的 Max_Payload_Size 参数通常为 128B、256B 或者 512B。

“Max_Payload_Size Supported” 字段仅表示该 PCIe 设备允许使用的 Max_Payload_Size 参数。在 Device Control 寄存器中，还有一个 Max_Payload_Size 参数，该字段可以由软件设置，表示实际使用的 Max_Payload_Size 参数大小。

值得注意的是，在 PCIe 设备中，“Max_Payload_Size Supported” 参数和 Max_Payload_Size 参数并不相同，前者是一个 PCIe 设备能够支持的最大 Payload 的大小，而后者是链路两端的 PCIe 设备进行协商，确定的实际使用值。有关这两个参数的详细说明见第 6.4 节。

该寄存器的第 4~3 位为 Phantom Functions Supported 字段，该字段只读。当 Device Control 寄存器的 Phantom Functions Enable 位为 1 时，该字段才有意义。

- 该字段为 0b00 时表示不支持 Phantom 功能，PCIe 设备不能使用 Function Number 扩展数据报文的 Tag 字段。
- 该字段为 0b01 时表示支持 Phantom 功能，PCIe 设备可以使用 Function Number 的最高

位扩展 TLP 的 Tag 字段。

- 该字段为 0b10 时表示支持 Phantom 功能，PCIe 设备可以使用 Function Number 的最高两位扩展 TLP 的 Tag 字段。
- 该字段为 0b11 时表示支持 Phantom 功能，PCIe 设备可以使用 Function Number 的全部三位扩展 TLP 的 Tag 字段。

该寄存器的第 5 位为 Extended Tag Field Supported 位，该位为 1 时表示 TLP 的 Tag 字段为 8 位；否则为 5 位。有关 Tag 字段的详细说明见第 6.3.1 节。本节不对该寄存器的其他位进行说明。

3. Device Control 寄存器

Device Control 寄存器各字段的描述如表 4-5 所示。

表 4-5 Device Control 寄存器

Bit	定 义	描 述
0	Correctable Error Reporting Enable	该位可读写，其复位值为 0。当此位为 1 时，PCIe 设备可以发出 ERR_COR Messages 报文。而当此位为 0 时，不支持这种操作
1	Non-Fatal Error Reporting Enable	该位可读写，其复位值为 0。当此位为 1 时，PCIe 设备可以发出 ERR_NON-FATAL Messages 报文。而当此位为 0 时，不支持这种操作
2	Fatal Error Reporting Enable	该位可读写，其复位值为 0。当此位为 1 时，PCIe 设备可以发出 ERR_FATAL Messages 报文。而当此位为 0 时，不支持这种操作
3	Unsupported Request Reporting Enable	该位可读写，其复位值为 0。当此位为 1 时，PCIe 设备可以发出 Unsupported Requests Error Messages 报文；而当此位为 0 时，不支持这种操作
4	Enable Relaxed Ordering	该位为 1 时，使能 PCIe 设备的 Relaxed Order 模式，即 PCIe 设备在发送 TLP 时，可以根据需要设置 TLP 的 Attr 字段为 Relaxed Ordering；该位为 0 时，TLP 的 Attr 字段不能设置为 Relaxed Ordering。该位复位时为 1，可读写
7:5	Max_Payload_Size	该字段可读写，PCIe 设备根据 Device Capability 寄存器的 Bit [2:0] 字段设置 PCIe 设备 TLP 的最大 Payload。系统软件根据 PCIe 链路两端的实际情况，确认该字段的值。但是该值不能大于 Device Capability 寄存器的“Max_Payload_Size Supported”字段 PCIe 设备发送 TLP 时，其最大 Payload 不能大于 Max_Payload_Size；当 PCIe 设备接收 TLP 时，必须能够处理小于该字段的 TLP，而大于该字段的 TLP 将被认做错误报文
8	Extended Tag Field Enable	该位为 1 时，发送端可以使用 8 位的 Tag 字段；该位为 0 时，可以使用 5 位的 Tag 字段。该字段的复位值为 1，可读写。Tag 字段的详细描述见第 6.3.2 节
9	Phantom Functions Enable	该位为 1，发送端可以使能 Phantom Function 功能；为 0，不使能这个功能。该字段的复位值为 0，可读写。Phantom Function 功能的详细描述见上文
10	Auxiliary (AUX) Power PM Enable	该位为 1 时，PCIe 设备可以使用总线提供的辅助电源
11	Enable No Snoop	此位为 1 时，PCIe 设备在发送 TLP 时，该 TLP 的 Attr 字段可以设置为 No Snoop；该位为 0 时，TLP 的 Attr 字段不能设置为 No Snoop。该位复位时为 1，可读写。该位与 Cache 共享一致性相关
14:12	Max_Read_Request_Size	该字段记录在一个 PCIe 设备中，存储器读请求 TLP 可以请求的最大数据区域。当 PCIe 设备发送存储器读请求 TLP 时，该 TLP 所请求的数据大小不能超过 Max_Read_Request_Size 参数 该字段的系统与表 4-4 中的描述相同

4. Device Status 寄存器

Device Status 寄存器主要字段的含义如表 4-6 所示。

表 4-6 Device Status 寄存器

Bit	定 义	描 述
0	Correctable Error Detected	该位为 1 时表示 PCIe 设备检测到 Correctable Error, 对该位写 1 将清除此位
1	Non-Fatal Error Detected	该位为 1 时表示 PCIe 设备检测到 Non-Fatal Error, 对该位写 1 将清除此位
2	Fatal Error Detected	该位为 1 时表示 PCIe 设备检测到 Fatal Error, 对该位写 1 将清除此位
3	Unsupported Request Detected	该位为 1 时表示 PCIe 设备收到一个 PCIe 总线并不支持的报文请求, 对该位写 1 将清除此位
4	AUX Power Detected	当 PCIe 设备检查到辅助电源的存在时, 而且如果该设备需要使用辅助电源, 则将该位置 1
5	Transactions Pending	对于 EP 而言, 该位为 1 表示当前 PCIe 设备发送了一个 Non-Posted 的数据请求, 但是没有收到完成报文应答; 对于 RC 和 Switch 而言, 该位为 1 表示 RC 和 Switch 自身 (并不是转发其他设备的 Non-Posted 数据请求) 发出了一个 Non-Posted 的数据请求, 但是没有收到完成报文应答

5. Link Capabilities 寄存器

Link Capabilities 寄存器描述 PCIe 链路的属性, 其主要字段的含义如下。

- Supported Link Speeds 字段。为 0b0001 表示 PCIe 链路支持 2.5 GT (gigatransfers)/s; 为 0b0010 表示 PCIe 链路支持 5 GT/s; 为 0b0100 表示 PCIe 链路支持 8 GT/s。
- Maximum Link Width 字段。该字段存放该 PCIe 设备支持的最大链路宽度。该字段为 0b000001 表示最大支持×1 的 PCIe 链路; 为 0b000010 表示最大支持×2 的 PCIe 链路; 为 0b000100 表示最大支持×4 的 PCIe 链路; 为 0b001000 表示最大支持×8 的 PCIe 链路; 为 0b001100 表示最大支持×12 的 PCIe 链路; 为 0b010000 表示最大支持×16 的 PCIe 链路; 为 0b100000 表示最大支持×32 的 PCIe 链路。
- ASPM (Active State Power Management) Support 字段, 该字段只读。0b00 和 0b10 为系统保留字段。当该字段为 0b01 时, 表示 ASPM 支持 L0s 状态; 当该字段为 0b11 时, 表示 ASPM 支持 L0s 和 L1 状态。PCIe 设备除了支持 PCI PM 电源管理方式之外, 还支持 ASPM 机制进行电源管理。ASPM 机制是 PCIe 设备进行的主动电源管理方式, 与系统软件没有直接联系。有关 ASPM 的详细描述见第 8.3 节。
- L0s Exit Latency 和 L1 Exit Latency 字段。这两个字段定义了 PCIe 设备从 L0s 和 L1 状态退出的最小延时。
- Port Number 字段。如果多端口 RC 和 Switch 支持多个下游端口, 则使用该字段对这些端口进行编号。PCIe 设备进行链路训练时, 需要使用这个端口号。

6. Link Control 寄存器

Link Control 寄存器主要字段的解释如下。

- ASPM Control 字段, 该字段可读写。该字段为 0b00 时表示禁止 PCIe 设备的 ASPM 机制; 为 0b01 时表示 PCIe 设备可以进入 L0s 状态; 为 0b10 时表示 PCIe 设备可以进入 L1 状态; 为 0b11 时表示 PCIe 设备可以进入 L0s 和 L1 状态。值得注意的是系统软件不能通过修改该字段使 PCIe 链路进入相应的状态, 仅是通知硬件逻辑, 可以进入相应的状态。ASPM 的详细描述见第 8.3 节。

- RCB(Read Completion Boundary)位。该位为 0 时,表示 RCB 为 64B,该位为 1 时,表示 RCB 为 128B。RCB 的大小与完成报文的有效负载相关。对于 RC 而言,该字段只读,而 Switch 和 EP 可以读写该字段。有关该位的进一步说明见第 6.4.3 节。
- Link Disable 位。向此位写 1,将禁止 PCIe 链路。此时链路状态机将进入 Disabled 状态,有关 PCIe 链路状态机的详细说明见第 8.2 节。
- Retrain Link 位。向此位写 1,将重新训练 PCIe 链路。此时 PCIe 链路状态机将进入 Recovery 状态。
- Common Clock Configuration 位,该位可读写。当该位为 1 时,表示 PCIe 链路两端的设备使用同源的参考时钟,即相同的 REFCLK 差分时钟;如果该位为 0,表示 PCIe 链路两端的设备使用的参考时钟并不同源,即使用异步时钟。
- Extended Sync 位,该位可读写。当该位为 1 时,表示 PCIe 设备退出 L0s 和进入 Recovery 状态时,需要额外发出一些同步序列。
- Hardware Autonomous Width Disable 位,该位可读写。当该位为 1 时,PCIe 设备不能改变当前已经协商好的 PCIe 链路宽度,除非为了修正 PCIe 链路中已经出现错误的 Lane。
- Link Bandwidth Management Interrupt Enable 位,该位可读写。当该位为 1 时,且 Link Status 寄存器的 Link Bandwidth Management Status 位为 1 时,PCIe 设备将向处理器提交中断请求。此时这个中断请求使用的中断向量由 PCI Express Capability 寄存器的 Interrupt Message Number 字段确定。
- Link Autonomous Bandwidth Interrupt Enable 位,该位可读写。当该位为 1 时,且 Link Status 寄存器的 Link Autonomous Bandwidth Status 位为 1 时,PCIe 设备将向处理器提交中断请求。

7. Link Status 寄存器

Link Status 寄存器存放 PCIe 设备正在使用的 PCIe 链路的状态,由链路宽度和速度等参数组成,其主要字段的含义如表 4-7 所示。

表 4-7 Link Status 寄存器

Bit	定 义	描 述
3:0	Current Link Speeds	为 0b0001 表示 PCIe 链路的传输率为 2.5 GT/s; 为 0b0010 表示 PCIe 链路的传输率为 5 GT/s; 为 0b0100 表示 PCIe 链路的传输率为 8 GT/s。该字段只读
9:4	Negotiated Link Width	该字段存放当前 PCIe 设备和其上游 PCIe 设备进行链路协商后使用的链路宽度。该字段为 0b000001 表示使用×1 的 PCIe 链路; 为 0b000010 表示使用×2 的 PCIe 链路; 为 0b000100 表示使用×4 的 PCIe 链路; 为 0b001000 表示使用×8 的 PCIe 链路; 为 0b001100 表示使用×12 的 PCIe 链路; 为 0b010000 表示使用×16 的 PCIe 链路; 为 0b100000 表示使用×32 的 PCIe 链路。该字段在 PCIe 链路进行训练的过程中,由硬件逻辑写入,系统软件只能读取该字段
11	Link Training	该位只读,为 1 时,表示 PCIe 链路正处于重新配置和重新训练阶段,当 PCIe 链路结束上述操作时,将此位清零
12	Slot Clock Configuration	该位由 PCIe 设备在初始化时确定,该位为 1 表示 PCIe 插槽与 Add-In 卡使用的参考时钟源相同。读者需要留意该位与 Common Clock Configuration 位的差别
13	Data Link Layer Link Active	该位表示 PCIe 链路的状态。该位为 1 时,表示 PCIe 链路处于 DL_Active,即正常工作状态

(续)

Bit	定 义	描 述
14	Link Bandwidth Management Status	该位由 PCIe 硬件设置。当 PCIe 链路重训练结束，或者 PCIe 设备完成 PCIe 链路的链路宽度和链路速度的设定后，该位置 1。该位写 1 清除
15	Link Autonomous Bandwidth Status	该位由 PCIe 设备确定。当 PCIe 链路自主完成链路宽度和速度的协商后，将该位置 1。该位写 1 清除

8. Device Capabilities 2 寄存器

该寄存器定义了一些 PCIe V2.1 总线规范使用的字段，其主要字段如表 4-8 所示。

表 4-8 Device Capabilities 2 寄存器

Bit	定 义	描 述
6	AtomicOp Routing Supported	Switch 的上下游端口和 RC 端口支持该位，在 PCIe V2.1 总线规范定义了原子操作。当该位为 1 时，表示原子操作 TLP 可以通过当前 Switch 或者 RC。有关原子操作的详细描述见第 6.3.5 节
7	32-bit AtomicOp Completer Supported	该位为 1 时，表示 EP 或者 RC 支持 32 位原子操作
8	64-bit AtomicOp Completer Supported	该位为 1 时，表示 EP 或者 RC 支持 64 位原子操作
9	128-bit CAS Completer Supported	该位为 1 时，表示 EP 或者 RC 支持 128 位原子操作。在 PCIe 总线规范中，只有 CAS (Compare and Swap) 支持 128 位操作
13:12	TPH Completer Supported	该字段为 0b00 时，表示接收端不支持 TPH 和扩展 TPH 报文；为 0b01 时，表示接收端仅支持 TPH 报文；为 0b11 时，表示接收端支持 TPH 和扩展 TPH 报文；而 0b10 保留。有关 TPH 的详细描述见第 6.3.6 节
20	Extended Fmt Field Supported	该位为 1 时，表示 TLP 的 Fmt 字段为 3 位，即支持 TLP Prefix；为 0 时，Fmt 字段为 2 位。Fmt 字段的详细描述见第 6.1.1 节。该位由 V2.1 规范引入，其目的是为了扩展 TLP 头
21	End-End TLP Prefix Supported	该位为 1 时，表示 EP 可以接收含有 End-End TLP Prefix 的 TLP
23~22	Max End-End TLP Prefixes	该字段限制一个 TLP 中 End-End TLP Prefix 的个数。该字段为 0b01 表示 TLP 中最多含有 1 个 End-End TLP Prefix；为 0b10 表示最多含有 2 个 End-End TLP Prefix；为 0b11 表示最多含有 3 个 End-End TLP Prefix；为 0b00 表示最多含有 4 个 End-End TLP Prefix

9. Device Control 2 寄存器

Device Control 2 寄存器主要字段的含义如表 4-9 所示。

表 4-9 Device Control 2 寄存器

Bit	定 义	描 述
6	AtomicOp Requester Enable	该位可读写，对 EP 和 RC 有效。如果该位和 Command 寄存器的“Bus Master Enable”位同时有效，EP 或者 RC 可以发出原子操作请求 TLP
7	AtomicOp Egress Blocking	该位可读写，对 Switch 的上下游端口和 RC 端口有效。当 AtomicOp Routing Supported 位为 1 时，该位可以为 1，否则该位只能为 0。该位为 1 时，Egress 端口将阻止原子操作 TLP 通过
8	IDO Request Enable	该位可读写，对 EP 和 RC 有效。当该位为 1 时，TLP 中的 IDO (ID-Based Ordering) 位可以根据实际情况（即 TLP 的 Attr2 位）设置为 1。IDO 是 PCIe V2.1 总线规范引入的新的“序”模型。有关 IDO 机制的详细说明见第 6.1.3 节

(续)

Bit	定 义	描 述
9	IDO Completion Enable	该位可读写。当该位为 1 时, EP 可以处理 IDO 位为 1 的完成报文
15	End-End TLP Prefix Blocking	该位可读写。当该位为 0 时, EP 不能发送带有 End-End TLP Prefix 的 TLP; 为 1 时, 可以发送

10. Root Control 和 Root Status 寄存器

这两个寄存器与 PCIe 总线的 AER (Advanced Error Reporting) 机制相关。其中 Root Control 寄存器由以下位组成。

- System Error on Correctable Error Enable。该位为 1 时, 表示 RC 端口管理的 PCI 树或者 RC 端口发送 ERR_COR 信息后, 将向处理器提交 System Error 信息。
- System Error on Non-Fatal Error Enable。该位为 1 时表示 RC 端口管理的 PCI 树或者 RC 端口发送 ERR_NONFATAL 信息后, 将向处理器提交 System Error 信息。
- PME Interrupt Enable。该位为 1 时, 如果 RC 端口收到 Root Status 寄存器的 PME Status 为 1 的信息后, 将向处理器提交 PME 中断信息。
- CRS Software Visibility Enable。当此位为 1 时, 系统软件发送配置请求 TLP 后, RC 端口可以要求该配置请求 TLP 择时重试。如当 PCIe 总线没有初始化完毕时, 不能接收处理器的配置请求, 此时将该位置 1; 初始化完毕后, 将该位置 0。

而 Root Status 寄存器由以下位和字段组成。

- PME Requester ID。该字段记录最后发送 PME 消息的 PCIe 设备的 Requester ID 号。
- PME Status。该位为 1 时, 表示 PCIe 设备 (ID 号为 PME Requester ID) 已经向 RC 发送了 PME 消息, 但是并没有被处理完毕, 该位写 1 清除。
- PME Pending。当 PME Status 位为 1 而且该位也 1 时, 表示 RC 中有尚未处理的 PME 消息。当 RC 清除 PME Status 位后, 硬件将向 RC 提交 PME 消息, 更新 PME Requester ID, 并将 PME Status 重新置为 1, 同时清除 PME Pending。PCIe 总线设置该位的主要目的是为了防止丢失 PME 消息。

PCI Express Capability 结构中还含有 Slot Capabilities、Slot Control 和 Slot Status 等寄存器。为节约篇幅, 本节并不对这些寄存器一一进行介绍, 在一个指定的 PCIe 设备中, PCI Express Capability 结构的这些寄存器并不会全部实现。许多 PCIe 设备甚至不存在 PCI Express Capability 结构, 但是在这些 PCIe 设备中依然存在与 PCI Express Capability 结构相关的概念, 只是这些结构没有以寄存器的形式表现出来, 供系统程序员使用而已。

4.3.3 PCI Express Extended Capabilities 结构

PCI Express Extended Capabilities 结构存放在 PCI 配置空间 0x100 之后的位置, 该结构是 PCIe 设备独有的, PCI 设备并不支持该结构。实际上绝大多数 PCIe 设备也并不支持该结构。在一个 PCIe 设备中可能含有多个 PCI Express Extended Capabilities 结构, 并形成单向链表, 其中第一个 Capability 结构的基址为 0x100, 其结构如图 4-19 所示。

在这个单向链表的尾部, 其 Next Capability Offset、Capability ID 和 Capability Version Number 字段的值都为 0。如果在 PCIe 设备中不含有 PCI Express Extended Capabilities 结构, 则 0x100

指针所指向的结构，其 Capability ID 字段为 0xFFFF，而 Next Capability Offset 字段为 0x0。

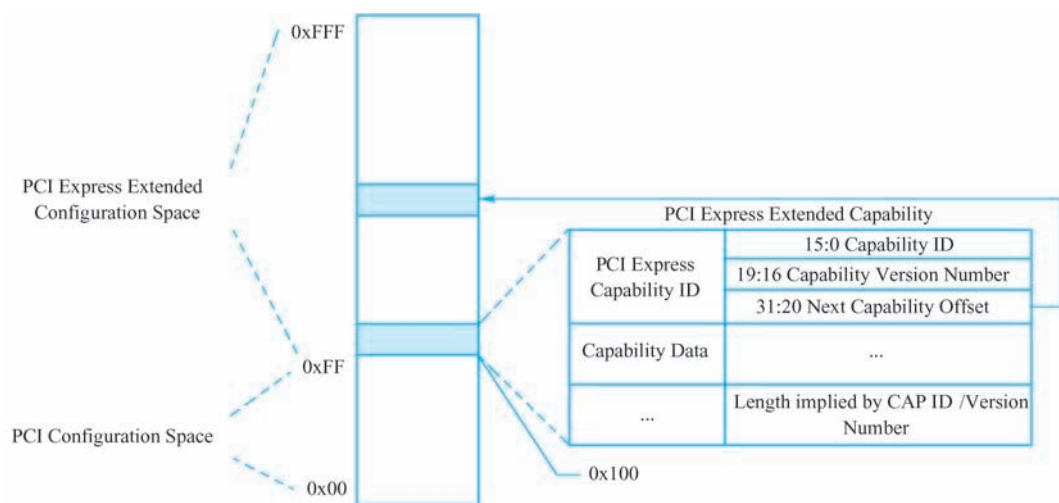


图 4-19 PCI Express Extended Capabilities 结构

一个 PCI Express Extended Capabilities 结构由以下参数组成。

- PCI Express Capability ID 字段存放 Extended Capability 结构的 ID 号。
- Capability Version Number 字段存放 Extended Capability 结构的版本号。
- Next Capability Offset 字段存放下一个 Extended Capability 结构的偏移。

PCIe 总线定义了一系列 PCI Express Extended Capabilities 结构，如下所示。

- AER Capability 结构。该结构定义了所有 PCIe 设备可能遇到的错误，包括 Uncorrectable Error（不可恢复错误）和 Correctable Error（可恢复错误）。当 PCIe 设备发现这些错误时，可以根据该寄存器的设置使用 Error Message 将错误状态发送给 Event Collector，并由 Event Collector 统一处理这些错误。系统软件必须认真处理每一个 Error Message，并进行恢复。对一个实际的工程项目，错误处理是保证整个项目可靠性的重要一环，不可忽视。AER 机制与 Error Message 报文的处理相关，第 6.3.4 节将进一步介绍 AER 机制。
- Device Serial Number Capability 结构。该结构记载 PCIe 设备使用的序列号。IEEE 定义了一个 64 位宽度的 PCIe 序列号，其中前 24 位作为 PCIe 设备提供商使用的序列号，而后 40 位由厂商选择使用。
- PCIe RC Link Declaration Capability 结构。在 RC、RC 内部集成的设备或者 RCRB 中可以包含该结构。该结构存放 RC 的拓扑结构，如 RC 使用的 PCI 链路宽度。如果 RC 支持多个 PCIe 链路，该结构还包含每一个链路的描述和端口命名。
- PCIe RC Internal Link Control Capability 结构。该结构的主要作用是描述 RC 内部互连使用的 PCIe 链路。该结构由 Root Complex Link Status 和 Root Complex Link Control 寄存器组成。
- Power Budget Capability 结构。当处理器系统为一些动态加入的 PCIe 设备分配电源配额时，将使用该设备的 Power Budget Capability 结构。
- ACS（Access Control Services）Capability 结构。该结构对 PCIe 设备进行访问控制管

理。RC 端口、Switch 的下游端口和多功能 PCIe 设备可以支持该结构。该结构与 PCIe 总线的 ACS 机制相关。ACS 机制定义了一组与收到的 TLP 相关的操作，该机制的原理较为简单，本节对此不做进一步分析。

- RCRB Header Capability 结构。该结构存放 RC 中的 RCRB，第 5.1 节将以 Montivina 平台为例介绍该结构的组成结构。
- RC Event Collector EP Association Extended Capability。在 x86 处理器系统中，RC 包含一个 Event Collector 控制器，该控制器处理 PCIe 设备发向 RC 的各类消息，如 PME 消息和 Error 消息。该结构用来描述 Event Collector 控制器。
- Multicast Capability 结构。PCIe 总线上的 RC、Switch 或者 EP 如果支持 Multicast 消息，需要使用该结构描述支持哪些 Multicast 组。PCIe 体系结构支持 Multicast 功能，在 PCIe 总线中，除了 PCIe 桥一定不支持 Multicast 功能外，其他设备都可以支持该功能。本节对 Multicast 功能不做进一步说明。

此外 PCIe 总线还可以支持其他 Capability 结构，如 Vendor-Defined Capability、Resizable BAR Capability、DPA（Dynamic Power Allocate）和 MFVC（Multi-Function Virtual Channel）Capability 结构等其他结构。但是在 PCIe 总线中，这些扩展的 Capability 结构并没有得到充分利用。在一个实际的 PCIe 设备中可能并不包含这些结构。

PCIe 设备定义的 Capability 结构有些过多，使用这种方法可以概括所有 PCIe 设备的使用特性。许多 PCIe 设备在支持这些 Capability 结构后，几乎可以不使用 BAR 寄存器空间存放与 PCIe 总线相关的任何信息。但是过多的 Capability 结构为软硬件工程师在设计上带来了不小的麻烦。一般说来，事务的发展过程是由简入繁，由繁化简。目前 PCIe 总线的发展仍处在由简入繁的过程。

本节仅详细介绍 PCI Express Extended Capabilities 结构组中的 MFVC 结构。MFVC 结构是 PCIe 总线的一个可选结构，其结构如图 4-20 所示。TLP 在通过 Switch 时需要通过 TC/VC Mapping，而且在进行 VC 仲裁和端口仲裁时，需要使用某些仲裁策略。

在 PCIe 总线中，TC/VC Mapping 表和 VC/端口仲裁策略在 MFVC 结构中定义。其结构如图 4-20 所示。值得注意的是，在许多 PCIe 设备中，可能只具有一个 VC，而且其 VC 仲裁的算法固定，那么在这个 PCIe 设备中，MFVC 结构并没有存在的必要。目前支持多 VC 的 PCIe 设备极少，仅有一些 RC 和 Switch 中存在多个 VC，而且也仅支持两个 VC。

VC Capability 的 ID 为 0x02 或者 0x09，VC Capability 结构由两部分组成，分别是一个 VC Capability 寄存器组和 n 个 VC Resource 寄存器组，其中 VC Resource 寄存器是可选的。如果 PCIe 设备仅支持一个 VC 时，该结构中不含有 VC Resource 寄存器，而在 VC Capability 寄存器组中包含该 VC 的描述信息。当一个 PCIe 设备支持 8 个 Function 时，则 n 为 7；如果支持 7 个设备，则 n 为 6，并以此类推。其中每一个 VC Resource 寄存器组中都包含一个 VC 仲裁表、端口仲裁表和 VC/TC 的映射表。

1. VC Capability 寄存器组

该组寄存器由 Port VC Capability Register 1、Port VC Capability Register 2、Port VC Control Register 和 Port VC Status Register 寄存器组成。

(1) Port VC Capability Register 1

该寄存器主要字段的含义如表 4-10 所示。

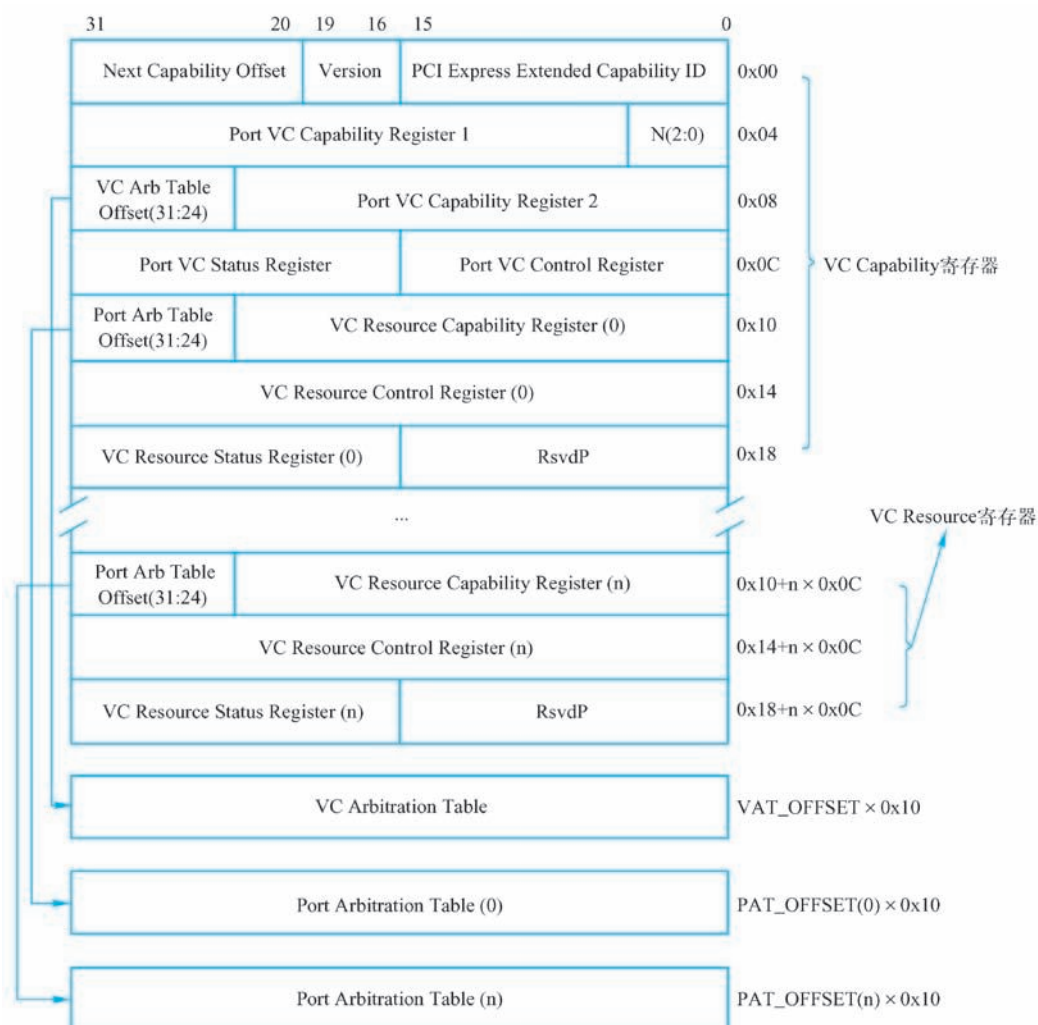


图 4-20 MFVC 结构

表 4-10 Port VC Capability Register 1

Bit	定 义	描 述
2:0	Extended VC Count	扩展的 VC 个数，最小值为 0，表示只支持 VC0；最大值为 7，表示支持 8 个 VC，VC0~VC7
6:4	Low Priority Extended VC Count	和 VC0 优先级相同的扩展 VC 的个数。在 PCIe 总线中，VC0 的级别最低。该字段的最小值为 0，最大值为 7
9:8	Reference Clock	如果 VC 使用 Time-based WRR 算法时，需要使用一个参考时钟。PCIe 总线规定当该字段为 0b00 时，这个参考时钟的周期为 100 ns，即时钟频率为 10 MHz
11:10	Port Arbitration Table Entry Size	表示 Port Arbitration Table Entry 的大小。0b00 表示 Entry 的长度为 1 位；0b01 表示 Entry 的长度为 2 位；0b10 表示 Entry 的长度为 4 位；0b11 表示 Entry 的长度为 8 位

(2) Port VC Capability Register 2

该寄存器由两个字段组成。其中 VC Arbitration Capability 字段存放 PCIe 设备支持的 VC 调度算法，PCIe 总线提供的调度算法包括 Hardware-fixed 仲裁策略和 WRR 仲裁策略。其中 WRR 仲裁策略分为 32、64 或者 128 个 Phase，VC 仲裁不支持 Time-based WRR 算法，有关

WRR 算法的详细说明见下文。

而 VC Arbitration Table Offset 字段存放 VC Arbitration Table 的地址偏移，如果在 PCIe 设备中不含有 VC Arbitration Table，该字段为 0。

(3) Port VC Control Register

该寄存器由 Load VC Arbitration Table 位和 VC Arbitration Select 字段组成。系统软件通过操纵该寄存器更改 VC 的仲裁算法，其中 VC Arbitration Select 字段用来选择 VC Arbitration Table 的长度，其关系如表 4-11 所示。

表 4-11 VC Arbitration Select 字段的说明

VC Arbitration Select 字段	VC Arbitration Table 的长度
0b001	32
0b010	64
0b011	128

Load VC Arbitration Table 位用来更新 VC 的仲裁算法。系统软件向 Load VC Arbitration Table 位写 1 时更新 VC 的仲裁算法，PCIe 设备可以根据 VC Arbitration Select 字段选择合适的仲裁算法，系统软件向 Load VC Arbitration Table 位写 0 没有意义。

(4) Port VC Status Register

Port VC Status Register 使用 VC Arbitration Table Status 位，控制更新 VC 仲裁算法的进度。当系统软件向 Load VC Arbitration Table 位写 1 时，PCIe 设备将更新 VC 的仲裁算法，并将 VC Arbitration Table Status 位置 1。PCIe 设备在没有完成仲裁算法的更换之前，VC Arbitration Table Status 位一直为 1，当 PCIe 设备完成仲裁算法的更换后，该位被 PCIe 设备清零。

2. VC Resource 寄存器组

在 PCIe 设备中，每一个 VC 都有一组 VC Resource 寄存器组，这组寄存器设置每一个 VC 的属性和端口仲裁算法。该组寄存器由 VC Resource Capability Register、VC Resource Control Register 和 VC Resource Status Register 寄存器组成。

(1) VC Resource Capability Register

该寄存器主要字段的含义如表 4-12 所示。

表 4-12 VC Resource Capability Register

Bit	定 义	描 述
7:0	Port Arbitration Capability	存放当前 VC 支持的端口仲裁算法。该字段对 Switch 和支持 Peer-to-Peer 传送的 RC 端口有效 Bit 0: 硬件固化的算法，如 RR Bit 1: WRR with 32 phases Bit 2: WRR with 64 phases Bit 3: WRR with 128 phases Bit 4: Time-based WRR with 128 phases Bit 5: WRR with 256 phases Bit 6~7: 保留
15	Reject Snoop Transactions	此位为 0 时，TLP 的 No Snoop 位无论是 0 还是 1 都可以通过该 VC；此位为 1 时，如果 TLP 的 No Snoop 位为 0，该 TLP 不能通过该 VC。该位对 RC 或者 RCRB 有意义

(2) VC Resource Control Register

该寄存器主要字段的含义如下。

- TC/VC Map 字段，第 7~0 位。该字段的每一位对应一个 TC，其中第 7 位对应 TC7，该位有效时表示 TC7 使用该 VC 进行数据传递；第 6 位对应 TC6，该位有效时表示 TC6 使用该 VC 进行数据传递，并以此类推。对于 VC0 通路，该字段的复位值为 0xFF，对于其他 VC 通路，该字段的复位值为 0x00。因此在系统初始化时，所有 TC 都使用 VC0 进行数据传递，而 PCIe 链路必须支持 VC0。使用该字段可以保证 TC 不同的 TLP 可以使用同一个 VC，但是在 PCIe 总线中，一个 TC 与一条 VC 建立了映射关系后，不能与其他 VC 建立映射关系。
- Load Port Arbitration Table 位，第 16 位。当该位被置 1 后，PCIe 设备将使用 Port Arbitration Table 更新端口仲裁的算法，当该位置 1 后，VC Resource Status 寄存器的 Port Arbitration Table Status 位也将置 1，当端口仲裁算法更新完毕后，Port Arbitration Table Status 位将清零；对此位写 1 没有意义。
- Port Arbitration Select 字段，第 19~17 位。该字段描述 Port Arbitration Table 表的 Entry 个数。下文将详细解释该字段。
- VC ID 字段，第 26~24 位。该字段存放当前 VC 的 ID 号，PCIe 设备的第一个 VC ID 必须为 0。
- VC Enable 位，第 31 位。该位为 1 时，当前 VC 通路有效，否则无效。在系统初始化完毕后，VC0 的 VC Enable 位为 1，而其他 VC 的 VC Enable 位为 0。

(3) VC Resource Status Register

该寄存器主要字段的含义如下。

- Port Arbitration Table Status 位，第 0 位。该位表示当前 VC 更新端口仲裁算法的状态，该位由 PCIe 设备维护，对系统软件只读。
- VC Negotiation Pending 位，第 1 位。该位为 1 表示当前 VC 通路正在进行初始化或者处于正在关闭的状态，此时当前 VC 并没有准备好，还没有从 FC_INIT2 状态中退出；为 0 表示当前 VC 准备好，PCIe 链路已经完成流量控制的初始化。系统软件必须保证该位为 0 后，才能对该 VC 进行操作。有关 FC_INIT2 状态的详细说明见第 9.3.3 节。

3. VC Arbitration Table

VC Arbitration Table 的长度由 Port VC Control 寄存器的 VC Arbitration Select 字段确定，最小为 32 个 Entry，最大为 128 个 Entry。VC Arbitration Table 实现 VC 仲裁的 WRR 算法。在 VC Arbitration Table 中，每一个 Entry 由 4 位组成，其中最高位保留，最低三位记录 VC 号。下文举例说明 32 个 Phase 的 WRR 算法，在这种情况下 VC Arbitration Table 的长度为 32，这个表中每一个 Entry 记录一个 VC 号，如图 4-21 所示。

在图 4-21 中，VC Arbitration Table 的每一个 Entry 都记录一个 VC 号。假定 VC 仲裁时从 Phase0 开始使用，该 Entry 存放的 VC 号为 VC0，则 VC 仲裁的结果是传送虚通路 VC0 中的总线事务，当这个总线事务传送结束后，将处理 Phase1 中的 VC；如果该 Entry 存放的 VC 号为 VC2，则 VC 仲裁的结果是传送虚通路 VC2 中的总线事务，并以此类推直到 Phase31 后，再对 Phase0 重新进行处理。

使用这种加权处理的方法，可以保证 PCIe 总线 QoS。值得注意的是，使用该方法时，如果当前 Entry 存放的 VC 中，不存在总线事务时，将迅速移动到下一个 Entry；如果 VC 间并没有出现冲突时，不需要使用该表进行仲裁，使用 64 个 Phase 和 128 个 Phase 的 WRR 算

法的实现机制与此类似。由上文的分析可以发现，与 RR 算法相比，PCIe 总线使用 WRR 算法可以在保证 QoS 的基础上，使各个 VC 公平使用端口资源。

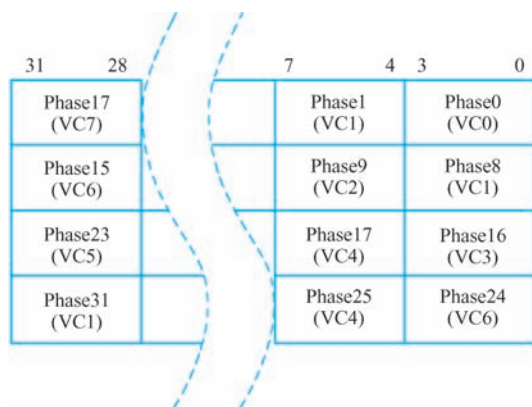


图 4-21 32 Phases 的 VC Arbitration Table

4. Port Arbitration Table

每一个 VC 都有一个 Port Arbitration Table，如图 4-21 所示。每一个 TLP 都首先需要进行端口仲裁之后，才能进行 VC 仲裁，然后通过端口发送。Port Arbitration Table 的主要作用是确定端口仲裁的策略。其长度由 VC Resource Capability Register 的 Port Arbitration Capability 字段确认，如表 4-12 所示。

在该表中，每一个 Entry 的大小由该设备支持的端口数目有关，如果一个设备支持 N 个端口，则该表 Entry 的大小为 $\lceil \log_2 N \rceil$ 。如果一个设备有 6 个端口，则 Port Arbitration Table 的 Entry 大小为 3。PCIe 总线支持 RR、WRR 和 Time-based WRR 端口仲裁策略。

Time-based WRR 端口仲裁策略的引入是为了支持 PCIe 总线的 isochronous 数据传送方式。在 PCIe 总线中使用 WRR 算法每处理完一个总线事务将移动一个 Phase，而 Time-based WRR 算法需要至少经过一个时间槽后才能移动一个 Phase。PCIe 总线为 Time-based WRR 算法使用的基准时钟周期在 Port VC Capability Register 1 的 Reference Clock 字段中定义，目前该值为 100ns。

PCIe 总线中使用的这些仲裁算法源于网络通信，这几种算法都是基于轮询的仲裁算法。在网络中，还经常使用 DWRR (Deficit Weighted Round Robin) 算法。

WRR 算法在支持长度不同的报文时，会出现带宽分配不公平的现象，为此 M. Shreedhar 与 George Varghese 提出了 DWRR 调度算法。DWRR 算法给每一个队列分配的权值不是基于报文的个数，而是基于报文的比特数。因此可以使各个队列公平地获得带宽。但是这种算法并不适用于 PCIe 总线，因为 PCIe 总线基于报文进行数据传递，而不是基于数据流。该算法在 ATM 分组交换网中得到了广泛的应用。

4.4 小结

本章简要介绍了 PCIe 总线的各个组成部件，包括 RC、Switch 和 EP 等，并介绍了 PCIe 总线的层次组成结构，和 PCIe 设备使用的 Capability 结构。本章是读者了解 PCIe 体系结构的基础。